

Ranking Opinions with Few States in Population Protocols

Tom-Lukas Breitkopf
Technische Universität Berlin
Berlin, Germany
t.breitkopf@tu-berlin.de

Antoine El-Hayek
Institute of Science and Technology Austria
Klosterneuburg, Austria
antoine.el-hayek@ist.ac.at

Julien Dallot
Technische Universität Berlin
Berlin, Germany
julien.dallot@tu-berlin.de

Stefan Schmid
Technische Universität Berlin
Berlin, Germany
stefan.schmid@tu-berlin.de

Abstract

Population protocols are a model of distributed computing where n agents, each a simple finite-state machine, interact in pairs to solve a common task against a (adversarial) interaction scheduler. This model was intensively studied in recent years; in particular, the problem of relative majority received much attention: Each agent starts with an input opinion (or color) out of k possibilities, and the goal is for each agent to eventually output the color with the largest support in the population. Before our work, the state complexity (the minimum number of states required per agent) was only known to be between $\Omega(k^2)$ and $O(k^7)$. Our main contribution is a population protocol that solves the relative majority problem with k^3 states. We achieve this result with a new protocol called CIRCLES. While prior approaches in the literature relied on duels of agents to find the majority color — an approach that proved effective for the case with two colors — CIRCLES partitions the agents into circular linked lists of decreasing sizes, with the property that no two agents with the same initial color lie in the same circle. We show that CIRCLES always correctly computes the desired structure against the most adversarial of schedulers (weakly fair). We then show that a trivial extension of CIRCLES solves the relative majority problem. We extend our protocol to handle various tie-breaking mechanisms or to support the case where the agents do not share a prior ordering of the colors. Finally, we show that a modification of CIRCLES solves the ranking problem with $2 \cdot k^4$ states, where each agent must output the rank of its initial color in the population.

CCS Concepts

• **Theory of computation** → *Distributed computing models*; • **Computing methodologies** → *Self-organization*.

Keywords

Population Protocols, Ranking, Relative Majority, Plurality Consensus

ACM Reference Format:

Tom-Lukas Breitkopf, Julien Dallot, Antoine El-Hayek, and Stefan Schmid. 2026. Ranking Opinions with Few States in Population Protocols. In *ACM*

Symposium on Principles of Distributed Computing (PODC '26), July 6–10, 2026, Egham, United Kingdom. ACM, New York, NY, USA, 18 pages. <https://doi.org/10.1145/3796701.3815913>

1 Introduction

Population protocols are a distributed computing model where a population of n finite-state agents interacts in a chaotic manner, updating their state according to a common protocol upon interacting with each other. Ever since their introduction in 2006 by Angluin et al. [7], population protocols have been widely studied and have become a standard setting to model key aspects of distributed computing [3]. Initially used as an abstraction for passively mobile finite-state sensor networks [7], population protocols features intriguing connections to natural processes [17], and can be used to model certain chemical reactions [19], dynamics in social groups or animal populations, or even gene regulatory networks [15]. The model further features connections to other theoretical models such as Petri nets, vector addition systems, semilinear sets and Presburger arithmetic [7]. Multiple problems have already been studied in this model, the most illustrative of which are the leader election and majority problem [23].

This paper focuses on two related problems in the population protocol model, *relative majority* and *ranking*. Both assume that initially every agent stores a color. Relative majority then asks for each agent to output the most popular color, i.e., the one with the largest initial support in the population. The ranking problem generalizes relative majority by asking each agent to output the position of its initial color ranked by popularity. Both problems are fundamental tasks in distributed computing [11] in order to ensure fault tolerance or to enable conflict resolution. The majority problem is also relevant outside computer science, with applications in physics, biology, statistics and sociology [13]. The ranking problem amounts to implementing a sorting algorithm in the population protocol model, which is a basic task in many subfields of computer science.

We are particularly interested in the fundamental question of how many states a protocol requires to solve a given task: the state complexity. This question is of relevance as in practice, the number of states that can be used is severely limited [4]: tiny sensor nodes have a limited memory [7], chemical applications ideally require a small number of chemical species involved in the reactions [18], and in DNA implementations the risk of faulty behavior increases with the number of states [4].



This work is licensed under a Creative Commons Attribution 4.0 International License. *PODC '26, Egham, United Kingdom*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2512-8/26/07

<https://doi.org/10.1145/3796701.3815913>

1.1 Related work

The majority problem is one of the most studied problems in the population protocol model, along with leader election [23]. Majority is a special case of relative majority with two colors: the agents can have one of two colors as input and the goal is for each agent to output the color with the larger support in the population. There exists a simple protocol solving majority with only 4 states per agent, concurrently found in [21] and [25]; this protocol is provably the best possible in terms of state complexity [5]. A lot of research focuses on the trade-off between time and state complexity [1, 2, 5, 12]. This culminated in a protocol with $O(\log n)$ many states, solving majority in expected $O(n \log n)$ interactions assuming in every step of the execution two agents interact uniformly at random. This protocol by Doty et al. [20] is optimal space-wise for protocols running in $O(n \log n)$ time.

A first study of multiple colors in a related model was conducted in 2015 by Salehkaleybar et al. [27]. They give solutions for both relative majority and ranking. Their definition of the ranking problem differs from ours, as they require every agent to output the whole ranking, not just the position of its own color in the ranking. Their algorithms require an exponential number of states: $O(k \cdot 2^k)$ states per agent for the relative majority problem and $O(k \cdot k!)$ states for the ranking problem, where k is the number of possible colors. They show that this bound is tight for the ranking problem – intuitively, an agent of any color (k many) should be able to output any possible ranking ($k!$ many). We note that the exponential state complexity of this protocol is not only implied by its output requirement, but also by its internal logic. The authors conjectured that also the bound on the state complexity of relative majority is tight. This conjecture was disproved in 2017 by Gąsieniec et al. [24] who first broke the exponential barrier and gave a protocol that solves the relative majority problem with $O(k^7)$ states (or as frequently cited $O(k^6)$ states in case the agents only have a binary output indicating whether their initial color is in relative majority). The protocol orchestrates a distributed tournament between colors: the initial colors of each agent are written in binary form and define a binary tournament tree according to the longest common prefix rule. The color that wins the tournament is deemed to be the majority color.

The above protocol relies on a hard-coded binary representation of the colors, which implies that the agents share a prior ordering on the colors. Motivated by biological use cases, Natale and Ramezani [26] proposed a modified model where no such prior ordering is known, allowing agents only to memorize colors and to compare them only for equality (a formal definition of a related model where agents are only able to store a single immutable color can be found in [14]). They proposed a protocol using $O(k^{11})$ states in the unordered model by combining the relative majority protocol of Gąsieniec et al. with another protocol to assign a numerical label to each color. Natale and Ramezani [26] also show that any protocol solving relative majority requires $\Omega(k^2)$ states.

Apart from the work listed above concerned with always-correct protocols, there is also research on protocols correctly converging with high probability. This relaxation allows providing protocols with $O(k)$ to $O(k + \log n)$ states, achieving different guarantees of validity depending on the initial bias between the majority opinion

Model	Reference	State comp.
Ordered	Salehkaleybar et al. [27] (2015)	$O(k \cdot 2^k)$
	Gąsieniec et al. [24] (2017)	$O(k^7)$
	This work	k^3
	Best-known lower bound [26]	$\Omega(k^2)$
Unordered	Natale and Ramezani [26] (2019)	$O(k^{11})$
	This work	$O(k^4)$

Table 1: Existing results on the state complexity to solve the relative majority problem with probability 1 and for k colors.

and the second most popular opinion [6, 9, 10, 22]. The additive bias required is typically $\Omega(\sqrt{n \log n})$ which motivates us in the search for an always-correct protocol.

1.2 Our contribution

This work presents a new always-correct protocol, CIRCLES, that can be adapted to solve both the relative majority and the ranking problems in the population protocol model. Unlike previous approaches for this problem [24, 26, 27], our protocol does not rely on pairwise duels between colors but rather on a novel way to structure the agents into circular linked lists, ensuring that the smallest lists contains colors with the greatest support. This approach enables us to improve upon the state of the art regarding the state complexity for both problems we consider. For relative majority, we achieve a state complexity of $O(k^3)$ under a weakly-fair scheduler (see Definition 2.1):

THEOREM 1.1 (THE CIRCLES PROTOCOL AND RELATIVE MAJORITY). *The CIRCLES protocol, which uses k^2 many states, denoted $\langle i|j \rangle$ for $i, j \in [k]$, stabilizes under a weakly-fair scheduler. In the stable configuration, agents of the form $\langle i|i \rangle$ exist if and only if i is the majority color. Propagating the result to every agent can be achieved by adding one output variable to the protocol, for a total of k^3 states.*

For the ranking problem, we break the exponential barrier [27] and provide a protocol that uses $O(k^4)$ under a globally fair scheduler (see Definition 2.2):

THEOREM 1.2. *There is a protocol that solves the ranking problem in case of a globally fair scheduler, and uses $2 \cdot k^4$ many states.*

Previous results combined several sub-protocols in an intricate way. In contrast, the CIRCLES protocol is surprisingly simple, making it easy to analyse and adapt to different use cases. In this work we not only use it to solve both the ranking problem and the relative majority problem, but also a variety of modifications of the relative majority problem, dropping the assumption that initially an ordering on the colors exists and handling ties in a number of different ways.

1.3 Organization

The remainder of this paper is structured as follows. In Section 2 we define the population protocol model, the relative majority problem in different versions and the ranking problem. In Section 3 we present our new protocol CIRCLES adapted to solve relative majority and prove its correctness. In Section 4 we present how to adapt CIRCLES to solve the relative majority problem while handling

ties in different ways. In Section 5 we show how to adapt CIRCLES to the unordered data model, where the agents do not share a prior ordering of the colors. In Section 6 we show how to adapt CIRCLES to solve the ranking problem, and in Section 7, we discuss the gap between the $O(k^3)$ upper bound and $\Omega(k^2)$ lower bound for the problem.

2 Model, Problems and Notations

We will now briefly introduce models and problems used within this work.

2.1 Models and Problems

The Population Protocol model. Unless stated otherwise, we use the standard notation from [7]. A *population* $\mathcal{P}$ consists of a set A of n agents and an irreflexive relation $E \subseteq A \times A$. It can be seen as a directed interaction graph. In this work we only consider the complete interaction graph. When a *population protocol* $\mathcal{A}$ runs in a population $\mathcal{P}$ every agent $a \in A$ is in one state of a finite set of states. The input function maps an agents input, taken from some finite input set, to an initial state. In this work we always consider an input set of k colors. After the initialization, pairs of agents interact. The agents have distinct roles when they interact, one is the *initiator* and the other is the *responder*. An interaction causes the initiator and the responder to update their states according to the transition function, which depends only on their previous states. Finally, the output of the population protocol is given by an *output function* which maps at any time an agent's state to an output, taken from a finite output set.

A *configuration* captures the “global state” of a population by assigning a state to each agent. Formally, a configuration can be fully described as a multiset of states. Similarly an input (or output) assignment maps an input (output) to every agent of the population. An interaction between two agents a, b in configuration S results in a new configuration S' , written as: $S \xrightarrow{(a,b)} S'$. Configuration S'' is *reachable* from S if there is some sequence of interactions, such that a population starting in S undergoing the interaction sequence results in S'' . We write this as $S \rightarrow S''$.

A *computation* of a protocol is an infinite fair sequence of interactions. Which two agents interact in each step is determined by a *scheduler*. We introduce two different schedulers formalizing two different notions of fairness as done in other work [8, 26].

Definition 2.1 (Weakly Fair Scheduler). In a computation under a *weakly fair scheduler* every pair of agents interacts infinitely often.

Definition 2.2 (Globally Fair Scheduler). In a computation under a *globally fair scheduler* for any two configurations S and S' with $S \rightarrow S'$ it holds that if S occurs infinitely often in the computation, then S' also occurs infinitely often.

A computation *stabilizes* to some output assignment when no future interaction can change the output of any agent.

The Unordered Data model. In the unordered data model, we assume that the agents do not agree on a prior ordering of the input colors: The agents can only compare their inputs for equality and

memorize colors. This setting was first used in [26] and a related version later formalized in [14].

Problems. In the literature the relative majority problem is also referred to as plurality, majority consensus or proportionate agreement [11]. The input assignment assigns a color from $\{0, 1, \dots, k-1\}$ to every agent. A protocol solving the problem must stabilize to an output assignment in which every agent indicates as output the color that has more supporters than any other. Different versions on how to handle ties will be introduced later.

In the ranking problem, the input assignment gives each agent a color from $\{0, 1, \dots, k-1\}$ as well. A protocol solving the problem must stabilize to an output assignment in which each agent indicates as output the position of its input color in a ranking of the colors: an agent with the plurality color as its initial color should output 1, an agent with the second most supported color should output 2, and so on.

2.2 Notations

We explain here the notations used throughout the paper. Proofs marked with $\star$ are deferred to the appendix.

multisets. For sets S and T we write S^T to denote the set of functions $f : T \rightarrow S$. If T is finite we call the elements of $\mathbb{N}^T$ *multisets* over T . In this paper, the subset $\subseteq$, the union $\cup$ and the set subtraction $\setminus$ operations are generalized to multisets: if $A, B \in \mathbb{N}^T$ are two multisets on T , it holds that $A \subseteq B$ if for all $t \in T$, $A(t) \leq B(t)$, we define $A \cup B$ as the multiset C such that for all $t \in T$, $C(t) = A(t) + B(t)$, we define $A \setminus B$ as the multiset C such that for all $t \in T$, $C(t) = \max(0, A(t) - B(t))$.

remainder. For $x \in \mathbb{Z}$ and $p \in \mathbb{N}^*$, we define $x \bmod p$ as the remainder of the euclidean division of x by p . Note that this is a number in $\mathbb{N}$, not $\mathbb{Z}/p\mathbb{Z}$.

ranges. Let $x, y \in \mathbb{N}$ such that $x \leq y$. $[x, y]$ denotes the set $\{x, x+1, \dots, y-1, y\}$, and (x, y) denotes the set $\{x+1, x+2, \dots, y-2, y-1\}$. With $[y]$ we denote $\{0, 1, \dots, y\}$.

modulo ranges. Let $x, y \in \mathbb{N}$. We define modulo ranges: $[x, y]_p$ denotes the set $\{x \bmod p, (x+1) \bmod p, \dots, (x+(y-x) \bmod p) - 1 \bmod p, (x+(y-x) \bmod p) \bmod p\}$, and $(x, y)_p$ denotes the set $\{(x+1) \bmod p, (x+2) \bmod p, \dots, (x+(y-x) \bmod p) - 2 \bmod p, (x+(y-x) \bmod p) - 1 \bmod p\}$. For instance, $[2, 7]_{10} = \{2, 3, 4, 5, 6, 7\}$ and $(8, 3)_{10} = \{9, 0, 1, 2\}$.

We say that a modulo range $[x, y]_p$ is included in another one $[x', y']_p$, denoted $[x, y]_p \subseteq [x', y']_p$ if it is included in the set-theoretic sense, with the extra condition: $x \neq y'$ or $y \neq x'$. This is to avoid the corner case: $[a, a+1]_p \subseteq [a+1, a]_p$, see Figure 1 for an illustration.

braket. We overload the braket notation, frequently used in quantum mechanics, to note ordered pairs. For $i, j \in \mathbb{N}$, we write $\langle i | j \rangle$ simply to distinguish the different roles of i and j . For agents storing some pair $\langle i | j \rangle$, we say they store a braket and refer to their bra and their ket for i and j respectively.

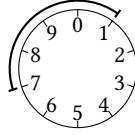


Figure 1: A modulo range contains all numbers covered by a clockwise walk from the start to the end of the range around the modular clock. This figure depicts $[7, 1]_{10}$.

3 Solving the Relative Majority Problem

3.1 Presentation of the protocol

In this section we introduce our protocol that solves the relative majority problem and prove its correctness. We consider the set $[0, k - 1]$ of k many colors and present below the CIRCLES¹ protocol.

States: The set of states Q contains every triples $(i, j, o) \in [0, k - 1]^3$. In the remainder of this paper we will use the bracket notation $\langle i|j \rangle$ to refer to the first two numbers of the triple, *bra* refers to $\langle i|$ and *ket* refers to $|j \rangle$, while *out* refers to o .

Input: each agent is initialized with $\langle i|i \rangle$ and $out = i$, where $i \in [0, k - 1]$ is the input color of the agent.

Output: return *out*

Transition function: We define *weights* for each bracket $\langle i|j \rangle$ as follows:

$$w(\langle i|j \rangle) = \begin{cases} k & \text{if } i = j \\ (j - i) \bmod k & \text{otherwise} \end{cases}$$

Two agents a and b that interact perform two successive operations:

- (1) a and b exchange their kets in case doing so strictly decreases the minimum weight of their two brackets.
- (2) If exactly one of a or b is of the form $\langle i|i \rangle$ for some $i \in [0, k - 1]$, set $out_a = out_b = i$.

The intuition of the protocol is depicted in Figure 2. Let's assume in this paragraph, for the sake of intuition, that only one color is in relative majority. As shown in Figure 2, think of a population as stacks of colors with the height of each stack representing the number of agents supporting it. If one were to remove rows from the stacks bottom to top until only a single row is left, then that row would contain only the color in relative majority. This is roughly what our protocol is doing, though it does not actually remove rows. Instead, in a way, it organizes the agents of a row into a cyclic linked list (which we call circle) and does not let them escape from it anymore. The *bra* of an agent represents its original color, while the *ket* of the agent represents the next color in the circle. The minimizing operation in the transition function ensures that if an agent is linked to another one too far away in the circle, it can always correct itself by meeting the correct *ket*. It organizes agents into a circle that is as tightly packed as possible (with respect to the numerical representation of their color). Once no agents can be added anymore, the circle never changes. The procedure then continues for the remaining agents, until in the final circle only one agent is contained: one holding the majority opinion.

¹We overload the notation slightly, and also refer by CIRCLES the protocol that behaves similarly but does not have an *out* entry. This is useful to use the protocol for other purposes than relative majority.

3.2 Proof of Correctness

We prove the protocol's correctness in Theorems 3.4 and 3.5. We beforehand introduce Greedy Independent sets, a construction on the input colors used to prove the protocol's correctness, as well as two necessary, fairly easy Lemmas 3.2 and 3.3. This is a formalisation of rows in Figure 2.

Definition 3.1 (Greedy Independent Sets). Consider the multiset of input colors to our protocol. We partition this multiset into sets $G_1, G_2, \dots, G_q$ as follows: store in G_1 as many inputs as possible as long as G_1 does not contain two equal colors; then apply the same on the remaining inputs to obtain G_2 , and so on.

We begin with this fairly straightforward observation:

LEMMA 3.2 (★). (Majority Color). Assume there exists a unique color μ in relative majority, then it holds that $G_q = \{\mu\}$ and there is no $j \neq \mu$ and $p \in [1, q]$ such that $G_p = \{j\}$.

We then introduce the main invariant of CIRCLES, which stems from the fact that agents only exchange kets.

LEMMA 3.3 (★). (Global Bracket Invariant). In every configuration and for all $i \in [0, k - 1]$, the number of bras $\langle i|$ and the number of kets $|i \rangle$ are equal.

THEOREM 3.4. (Stabilization). Running CIRCLES, the agents exchange their kets a finite number of times.

PROOF. We call ω the smallest ordinal number greater than all the integers². We prove the claim by exhibiting a quantity that strictly decreases at each ket exchange. Given a configuration C , let $w_1(C), w_2(C) \dots w_n(C)$ be the weights of all the brackets sorted in increasing order. Define

$$g(C) = \omega^{n-1} \cdot w_1(C) + \omega^{n-2} \cdot w_2(C) + \dots + \omega \cdot w_{n-1}(C) + w_n(C)$$

Assume that two agents exchange their kets. Let p be the lowest index such that $w_p(C)$ changes before and after the ket exchange. By design of the protocol, $w_p(C)$ strictly decreases. This implies that g strictly decreases when two agents exchange their kets. As an ordinal number cannot decrease infinitely many times, the number of ket exchanges is therefore finite. $\square$

THEOREM 3.5. (Correctness). Assume that there exists a unique color μ in relative majority. In the CIRCLES protocol, all agents eventually output μ under a weakly fair scheduler.

We define the brackets associated with a Circle:

Definition 3.6 (Circle Bracket Sets). For a given greedy independent set G_p with $p \in [1, q]$ (Definition 3.1), let $g_0, g_1, \dots, g_m$ be the elements of G_p sorted in increasing order and define

$$f(G_p) = \{\langle g_0|g_1 \rangle, \langle g_1|g_2 \rangle, \dots, \langle g_m|g_0 \rangle\}$$

Theorem 3.5 follows directly from the following lemma:

LEMMA 3.7. After Stabilization (Theorem 3.4), let C be the multiset of brackets of the agents. We have that:

$$C = \bigcup_{p=1 \dots q} f(G_p)$$

²For readers unfamiliar with ordinals, we recommend checking this wikipedia page, or this 15-minute video, which contains all that is needed for our proofs.

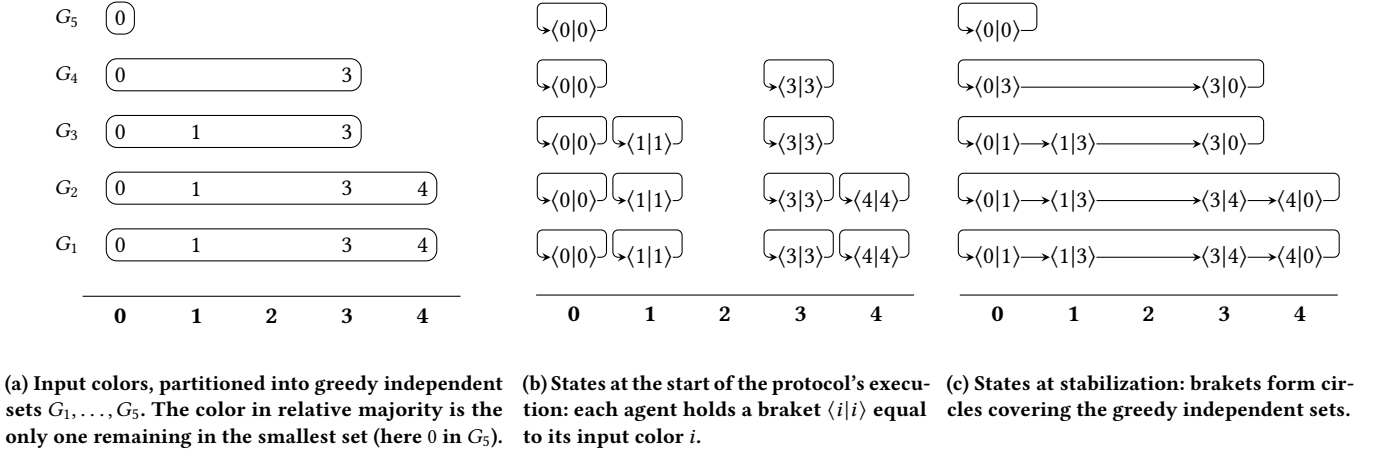


Figure 2: Example of CIRCLES with $k = 5$ colors and $n = 14$ agents. Agents with the same input color are stacked vertically.

PROOF. We prove the following predicate $H(r)$ by induction on $r \in [0, q]$:

$$\bigcup_{p=1 \dots r} f(G_p) \subseteq C \quad (H(r))$$

The base case for $r = 0$ is trivial.

Let $r \in [0, q - 1]$, we assume that $H(r)$ holds and show that $H(r + 1)$ also holds. We define the subconfiguration $C[r + 1] = C \setminus \bigcup_{p=1 \dots r} f(G_p)$.

Case 1: $\bigcup_{p=r+1}^q G_p$ contains only elements from one color. Let i be that color.

Then for any other color $j \neq i$, there are at most r many bras $\langle j|$ and as many kets $|j\rangle$, which are all included in $\bigcup_{p=1 \dots r} f(G_p)$. Thus all agents in $C[r + 1]$ are of the form $\langle i|i \rangle$. Since $\{\langle i|i \rangle\} = f(G_{r+1})$, we have $\bigcup_{p=1 \dots r+1} f(G_p) \subseteq C$.

Case 2: There are at least two different colors in G_{r+1} .

We note $g_0, g_1, \dots, g_m$ the elements of G_{r+1} sorted in increasing order. Let $l \in [0, m]$. To lighten notations, we will mean $l + s \bmod (m + 1)$ each time we write $l + s$ in the remainder of this proof. We prove that, if there is no agent with bracket $\langle g_l|g_{l+1} \rangle$ in $C[r + 1]$, then there exist two agents whose interaction creates that bracket, a contradiction with the stability hypothesis. First notice that $\langle g_l|$ and $|g_{l+1} \rangle$ are in $C[r + 1]$. Indeed, note that there are at least $r + 1$ many $\langle g_l|$ and $r + 1$ many $|g_{l+1} \rangle$ in C , as there are at least $r + 1$ many agents with color g_l and g_{l+1} initially. By Theorem 3.3, this means we have at least $r + 1$ many $|g_{l+1} \rangle$ in C . Because each f_p for $p \leq r$ contains exactly one $\langle g_l|$ and one $|g_{l+1} \rangle$, we have that both $\langle g_l|$ and $|g_{l+1} \rangle$ are in $C[r + 1]$.

Assuming by contradiction that there is no agent with bracket $\langle g_l|g_{l+1} \rangle$ in $C[r + 1]$, then there exists an agent with bracket $\langle g_l|j \rangle$ and an agent with bracket $\langle i|g_{l+1} \rangle$ in $C[r + 1]$ for some i and j . We show that those two agents exchange their kets if they interact.

CLAIM 1. $i, j \notin (g_l, g_{l+1})_m$

PROOF. By contradiction, assume that i is in $(g_l, g_{l+1})_m$. A $\langle i|i \rangle$ in $C[r + 1]$ indicates that i had initially at least $r + 1$ agents supporting it, as by contradiction if it was not the case, all the bras $\langle i|$ would

have been in $\bigcup_{p=1}^r f(G_p)$. By construction of G_{r+1} , we must have that i is in G_{r+1} , and thus, g_l and g_{l+1} are not consecutive in the ordered list of G_{r+1} , a contradiction. The case $j \in (g_l, g_{l+1})_m$ is symmetric. $\square$

If $i \neq g_{l+1}$, it holds by Claim 1 that

$$\begin{aligned} w(\langle g_l|g_{l+1} \rangle) &= (g_{l+1} - g_l) \bmod k < (g_{l+1} - i) \bmod k \\ &= w(\langle i|g_{l+1} \rangle) \end{aligned}$$

Otherwise, if $i = g_{l+1}$:

$$w(\langle g_l|g_{l+1} \rangle) = (g_{l+1} - g_l) \bmod k < k = w(\langle i|g_{l+1} \rangle)$$

Similarly, if $j \neq g_l$, it holds by Claim 1 that

$$w(\langle g_l|g_{l+1} \rangle) = (g_{l+1} - g_l) \bmod k < (j - g_l) \bmod k = w(\langle g_l|j \rangle)$$

Otherwise, if $j = g_l$:

$$w(\langle g_l|g_{l+1} \rangle) = (g_{l+1} - g_l) \bmod k < k = w(\langle g_l|j \rangle)$$

Which proves that exchanging kets between $\langle g_l|j \rangle$ and $\langle i|g_{l+1} \rangle$ reduces the minimum weight. An interaction between the two agents eventually happens as the scheduler is weakly fair, this interaction would therefore trigger a ket exchange, which is in contradiction with the stability hypothesis: we deduce that $\langle g_l|g_{l+1} \rangle \in C[r + 1]$, which implies $f(G_{r+1}) \subseteq C[r + 1]$ and therefore $H(r + 1)$ holds.

We proved by induction that $H(q)$ holds:

$$\bigcup_{p=1 \dots q} f(G_p) \subseteq C$$

As $|\bigcup_{p=1 \dots q} f(G_p)| = |C|$ we can rewrite $H(q)$ as an equality and the claim holds. $\square$

PROOF OF THEOREM 3.5. By Lemma 3.7 and Lemma 3.2, after Stabilization (Theorem 3.4), since we assumed that there is only one majority color, there exists at least one agent in bracket $\langle \mu|\mu \rangle$ and none in bracket $\langle j|j \rangle$ for $j \neq \mu$. The agent(s) with bracket $\langle \mu|\mu \rangle$ will transmit their output color μ to the rest of the population and the claim follows.

The claim on the number of states needed is straightforward, as 3 numbers between 0 and $k - 1$ are stored. In the case where agents

only need to output whether or not their color is the majority one, an extra bit needs to be stored to record that fact, and the $\langle i|i \rangle$ agents need to flip it accordingly when meeting other agents. $\square$

This yields the following theorem:

THEOREM 1.1 (THE CIRCLES PROTOCOL AND RELATIVE MAJORITY). *The CIRCLES protocol, which uses k^2 many states, denoted $\langle i|j \rangle$ for $i, j \in [k]$, stabilizes under a weakly-fair scheduler. In the stable configuration, agents of the form $\langle i|i \rangle$ exist if and only if i is the majority color. Propagating the result to every agent can be achieved by adding one output variable to the protocol, for a total of k^3 states.*

Note that before this last proof, at no point did we use the fact that there is only one majority color, and thus, we can use all the lemmas to analyze what happens in the case of ties, which we do in the following sections.

3.3 A note on the time complexity

While this paper focuses on the space complexity of the problem, it is worthwhile talking briefly about the time complexity of the problem, which is typically done under the *random scheduler*: where for each interaction, two agents are chosen uniformly at random. Before talking about the CIRCLES protocol, let's look at an intermediate result: Assume you have ℓ pairs of tokens on some agents that need to meet, in any order. Then in any interaction, the probability that one such pair is chosen is $\frac{2\ell}{n(n-1)}$, and thus the expected time until one such pair being chosen is $\frac{n(n-1)}{2\ell}$. Then there are only $\ell - 1$ many pairs to interact. Thus, the total expected time for all pairs to interact is $\sum_{\lambda=1}^{\ell} \frac{n(n-1)}{2\lambda} = \Theta(n^2 \log \ell)$.

Now consider the CIRCLES protocol. Assume for the moment that all opinions initially have the same number of agents. Then there are n bra-ket pairs that need to meet before stabilisation, in any order. By the result above, this takes $O(n^2 \log n)$ time.

If we remove the assumption that all opinions initially have the same number of agents, then order matters. Indeed, the circles with more agents need to have stabilised before circles with fewer agents can start pairing up. There are at most k many different circles, and thus, the total running time is at most $O(kn^2 \log n)$.

4 Tie Handling Mechanisms

In this section, we describe three different ways to handle ties in the relative majority problem.

4.1 Reporting Ties

We present the CIRCLES_TIEREPORT protocol which extends the CIRCLES protocol and enables it to report if there is more than one color in relative majority, i.e. if there is a tie. CIRCLES_TIEREPORT solves a version of the relative majority problem where, in case of a tie, all the agents must return a specific value to indicate that there is a tie. This feature was present in the paper from Gąsieniec et al. [24] where the agents could keep track of a potential tie with the label zero. In case there is no tie, all the agents must still return the majority color.

We add to the agents' states a variable `tie_state` on top of the states of CIRCLES. `tie_state` can have three values: `no_tie`, `tie` or `tie_spreader`. At the beginning, all the agents are at `no_tie`. Without

interfering, `tie_state` evolves in parallel of CIRCLES and follows three rules.

- (1) **tie detection.** An agent $\langle i|i \rangle$ that decays into $\langle i|j \rangle$ with $i \neq j$ goes to `tie_spreader`.
- (2) **tie spreading.** An agent at `tie_spreader` sets to `tie` any encountered agent $\langle i|j \rangle$ such that $i \neq j$.
- (3) **tie correction.** An agent $\langle i|i \rangle$ sets any encountered agent to `no_tie`.

The output changes as follows:

- Agents in `no_tie` output the winning color stored in `out` as usual (by default behavior of CIRCLES).
- Agents in `tie` or `tie_spreader` output "tie".

THEOREM 4.1 (★). *CIRCLES_TIEREPORT solves the relative majority problem, where all agents output the color of the majority if the majority is strict. If there is a tie, agents do not output a color, but rather report that there is a tie. It requires $3k^3$ many states, which can be reduced to $4k^2$ if agents are not required to output the majority color, but only whether their color wins or loses.*

4.2 Sharing Ties

In this subsection we would like that, even in case of a tie, all agents with a winning color output their color. Agents with a losing color output any winning color.

The CIRCLES_TIESHARING protocol is defined as follows: From the CIRCLES, we keep everything apart from item (2) of the transition function (the one that updates the `out` of agents). We add a variable "status" that can have values `winner_spreader`, `winner` or `loser`. All agents start at `winner`. The variable status evolves in parallel of CIRCLES and follows the following rules, where all brackets below are considered after the application of the transition rule (1) of CIRCLES.

- (1) **token generation:** An agent that exchanged kets to become $\langle i|j \rangle$ with $i \neq j$ is set to `winner_spreader`.
- (2) **token destructor:** An agent $\langle i|j \rangle$ that recombines into $\langle i|i \rangle$ is set to `winner`.
- (3) **token destruction:** An agent with bracket $\langle i|i \rangle$ sets agents at $\langle i|j \rangle$ to `winner`, otherwise sets agents at $\langle i'|j \rangle$ with $i' \notin \{i, j\}$ to `loser`.
- (4) **token multiplication:** If a with $\langle i|j \rangle$ at `winner_spreader` meets b at `loser` with $\langle j|j' \rangle$, set b to `winner_spreader`.
- (5) **token ascent:** If an agent a with $\langle i|j \rangle$ at `winner_spreader` meets an agent b with $\langle i'|j' \rangle$ such that $[i, j]_k \subseteq [i', j']_k$, set b to `winner_spreader` and a to `winner` if $i = i'$ and to `loser` otherwise.
- (6) **token simplification:** If an agent a with $\langle i|j \rangle$ at `winner_spreader` meets an agent b at `winner_spreader` with $\langle i'|j' \rangle$ such that $[i, j]_k \subseteq [i', j']_k$, set a to `winner` if $i = i'$ and to `loser` otherwise.
- (7) **information spreading:** If an agent a with $\langle i|j \rangle$ at `winner_spreader` meets an agent b with $\langle i|j' \rangle$, set b to `winner`. If b is of the form $\langle i'|j' \rangle$ with $i' \neq i$, and $[i', j']_k \subseteq [i, j]_k$, set b to `loser`.

In parallel, the `out` value of agents is also updated:

- (i) If an agent a with $\langle i|i \rangle$ meets an agent b of the form $\langle i'|j' \rangle$ with $i' \neq j'$, set `outb` to i .

- (ii) If an agent a with $\langle i|j \rangle$ at *winner_spreader* meets an agent b of the form $\langle i'|j' \rangle$ with $[i', j']_k \subseteq [i, j]_k$, set out_b to i .

We say that an agent has a token if it is at *winner_spreader*, and doesn't have a token if it is at either *winner* or *loser*.

THEOREM 4.2 (★). *Let $M = \mu_0, \mu_1, \dots, \mu_{r-1}$ be the colors in relative majority. In the CIRCLES_TIESHARING protocol, eventually all agents with initial color in M are at *winner_spreader* or *winner* while all other agents are at *loser*. Moreover, the output of every winner or *winner_spreader* is its initial color, while the output of loser agents is one of the winning colors.*

The protocol uses $3k^3$ many states. This can be dropped to $3k^2$ if we only require each agent to only output whether their color win or lose.

4.3 Breaking ties

Instead of reporting or sharing ties it is also possible to break them, meaning in case of a tie all agents agree on one of the colors in relative majority. We adapt CIRCLES by adding a single bit *tie_break_token* (short *tbt*) and by running additional steps in parallel to CIRCLES. We call the resulting protocol CIRCLES_TIEBREAK. We say that an agent a carries the tie-break token if $tbt_a = 1$, while agents with $tbt = 0$ do not. The idea is as follows: whenever two agents exchange their kets, a tie-breaking token is generated. When two tokens meet, one token is destroyed. The token is always passed along to the bracket with the largest weight. The idea we rely on is that in case of a tie the bracket with largest weight is formed of the bra and the ket of two majority colors. In case there is no tie, as before an agent with $\langle i|i \rangle$ for the unique majority color $i \in [0, k-1]$ exists, obtaining the maximum weight of k .

More precisely the CIRCLES_TIEBREAK protocol is obtained from the CIRCLES protocol by executing the following steps in sequence after applying the transition function of CIRCLES. Note that agents may apply multiple steps (e.g. (2) and (3)) in a single interaction.

- (1) **token generation:** If the agents exchange their kets (due to CIRCLES) and neither one currently holds a tie-break token, then one of them generates a token (sets tbt from 0 to 1).
- (2) **token destruction:** If both agents hold a tie-break token, then one of them destroys its token (sets tbt from 1 to 0).
- (3) **token transfer:** If exactly one agent holds a tie-break token, then they pass the token to the one with the larger weight. In case the weight of the agents is identical, then they pass the token to the one with the larger bra (to pass a token from a to b , set tbt_a from 0 or 1 to 0 and tbt_b from 0 or 1 to 1).

After applying the rules above, the output of agents is set:

- If agent a has the token, set $out_a = out_b = i_a$, where a has bracket $\langle i_a | j_a \rangle$.

In case we do not specify which of the agents should generate or destroy a token, it does not matter for the correctness of the protocol, so w.l.o.g. we chose the initiator.

THEOREM 4.3 (★). *CIRCLES_TIEBREAK solves the relative majority problem against a weakly-fair scheduler. If there is a tie, agents all output the same color, one of the majority ones.*

It requires $2k^3$ many states, which can be reduced to $3k^2$ if agents are not required to output the majority color, but only whether their color wins or loses.

5 Unordered setting

The CIRCLES protocol, which we introduce in this work to solve relative majority, relies on numerical representations of the colors in order to compute a distance between them. In this section we describe how to adapt CIRCLES to the unordered setting, in which agents are only able to compare colors for equality and to memorize them. We use the overall idea proposed by Natale and Ramezani [26] which is to combine our protocol with an ordering protocol that assigns to all agents of some initial color from color set C the same numeric label from label set $K = [k-1]$. We, however, use another ordering protocol, that is more memory-efficient.

5.1 The ORDERING protocol

The protocol reuses some ideas proposed by Natale and Ramezani [26] and combines them with a modified version of the protocol proposed by Cai et al. [16], solving (a variant of) the leader election problem. Initially all agents receive label 0. ORDERING elects one leader per color among the agents initially supporting that color (by using the asymmetry of interactions). Whenever two leaders with different colors meet (recall that the agents can check for equality between colors), then one of them increments its label by one modulo k in case they have the same label. Non-leaders simply copy the label of their leader. Eventually there is a unique leader per color and all leaders store a different label, that they then distribute to the non-leaders.

THEOREM 5.1 (★). *ORDERING solves the ordering problem with $O(k^2)$ states.*

Intuitively the proof of correctness associates a potential function with every configuration that measures how far away the configuration is from containing a valid ordering. It decreases by a lot in case a leader chooses a previously unused label and by a little in case a leader gets closer to an unused label, until it eventually reaches 0 and an ordering is established.

5.2 The ORD-CIRCLES protocol

The output of ORDERING is used as input to CIRCLES. To reiterate: ORDERING computes a numeric *label* from set $K = [k-1]$ for an agents initial *color* from (the arbitrary) set C . That label is stored as the agents *bra*, which functions as input to CIRCLES. One important property that is required for the correctness of CIRCLES is that it maintains the invariant from Lemma 3.3, so for every bra $\langle i|$ in the population for some $i \in K$, there needs to be a matching ket $|i \rangle$ in the population as well. Care must therefore be taken when overwriting the bra of an agent with a new label. Whenever ORDERING wants to update an agents bra, we therefore make sure that it is done in a way that maintains the invariant. We say that an agent is *consistent* in case its bracket is $\langle i|i \rangle$ for $i \in K$ and *inconsistent* otherwise. Before applying changes to an agent's bra, we first wait for the agent to become consistent. Then its bracket can be re-initialized to a new value $\langle j|j \rangle$ and Invariant 3.3 still holds. The idea of waiting for agents to become consistent before transmitting changes of the ordering protocol to the relative majority protocol also originates from Natale and Ramezani [26].

In order to remember what changes ORDERING wants to apply to the bra of an agent until that agent becomes consistent, we introduce an *ordering state*. The ordering state stores whether ORDERING wants to increment the bra, copy it from the leader or leave it unchanged. For that it can take one of the values I, CL, CLW or U (for increment, copy leader, copy leader wait or unchanged). The two separate states CL and CLW are required to make the following distinction: an agent in ordering state CLW wants to copy the label of its leader, but first has to wait to become consistent, while an agent in ordering state CL is already consistent and now only has to wait to meet its leader in order to copy its label. Agents in ordering state U will be called *stable*, because they do not want to make any changes to their bra. Agents in I or CLW will be called *unstable* to indicate they want to change their bra, but first need to become consistent. Finally agents in CL we call *pending*, highlighting that they are already consistent and only waiting to meet their leader.

Formally the ORD-CIRCLES protocol which solves relative majority in the unordered setting works as follows.

Memory organization and initialization: Agents store all memory entries related to CIRCLES: $\langle i|j \rangle$, *out* and depending on the use case bits to break, share or report ties. Additionally there are memory entries *ic* to store the initial color of an agent (in some representation) and the leader bit *ld* required for ORDERING. Agents also store an *ordering state* $os \in \{U, I, CL, CLW\}$. Initially $i = j = 0$, $os = U$ and $ld = 1$. Both *ic* and *out* are initialized with the input color of the agent.

Transition function: When we say an agent with bracket $\langle i|i \rangle$ re-initializes for some value i' , then it sets its bracket to $\langle i'|i' \rangle$, $os = U$, $out = ic$ and re-initializes all variables related to reporting or breaking ties as described in the respective sections. An interaction between agents a and b , with brackets $\langle i_a|j_a \rangle$ and $\langle i_b|j_b \rangle$, results in state updates by performing the following steps in sequence:

- (1) In case a and b are leaders of the same color, then a stops being a leader, meaning it sets $ld_a = 0$. In case b is the leader of a , $i_a \neq i_b$ and a is not pending, then a sets $os_a = CLW$. In case a and b are leaders of different colors, have the same bra and b is stable, then a sets $os_a = I$. This step corresponds to ORDERING.
- (2) If a is pending, a and b have the same initial color $ic \in C$ and b is a leader, then a re-initializes its label with value i_b . Pending agents do otherwise, not interact in any of the other steps.
- (3) If a is unstable and inconsistent, and $j_b = i_a$, then a and b exchange their *ket*, making a consistent.
- (4) If a is unstable and consistent, then in case $os_a = I$, a re-initializes for the value $(i_a + 1) \bmod k$. In case $os_a = CLW$, then a sets $os_a = CL$.
- (5) a and b interact according to CIRCLES.

In order for agents to output elements from the set of colors C instead of elements from the set of labels K , manipulate CIRCLES as follows: Whenever some agent a sets $out_a = \langle i_b| \rangle$ for some agent b (potentially $a = b$), instead set $out_a = ic_b$.

THEOREM 5.2 (★). *ORD-CIRCLES solves relative majority in the unordered setting against a weakly-fair scheduler using $O(k^4)$ states.*

This can be reduced to $O(k^3)$ many states if every agent is only required to report whether their color wins or loses.

The proof of correctness essentially argues that eventually an ordering among the colors is established and that afterwards CIRCLES runs on a population correctly representing the initial color distribution with the bra and ket of agents.

6 The Ranking protocol

In this section we discuss how to adapt the CIRCLES protocol to solve the ranking problem under a globally fair scheduler.

For the rest of this section, we denote by x_i , for each $i \in [0, k-1]$, the number of agents with color i in the input.

Definition 6.1. In the *ranking problem*, each agent is given as an input a color in $[0, k-1]$. We assume that no two colors have the same number of agents initially, and that all colors are represented at least once. At the end of the protocol, each agent with input i should output a value $j \in [0, k-1]$, such that $|\{\ell \in [0, k-1] : x_\ell > x_i\}| = j$, which corresponds to the ranking of the color in the input, that is, the majority color outputs 0, the second most popular color outputs 1, and so on.

THEOREM 1.2. *There is a protocol that solves the ranking problem in case of a globally fair scheduler, and uses $2 \cdot k^4$ many states.*

Remark 1. *This algorithm can be easily modified to work for the case where the colors are unordered, with the same number of states.*

Let us first discuss the intuition and observations that lead us to such a protocol. First, in the CIRCLES protocol, we notice that, in the very specific case where color 0 is the most popular one, followed by 1, then 2, and so on until $k-1$, then the structure of the configuration at stabilization is very straightforward: each set of the Greedy Independent Sets is of the form $[0, i]$ for $i \in [0, k-1]$. In turn, this means that all of the brackets are of the form $\langle i|i+1 \bmod k \rangle$ or $\langle i|0 \rangle$ for $i \in [0, k-1]$. In fact, we can show that this is a if and only if situation:

LEMMA 6.2. *In the CIRCLES protocol, the input is such that $x_0 > x_1 > \dots > x_{k-1}$ if and only if, after stabilization, all of the brackets are of the form $\langle i|i+1 \bmod k \rangle$ or $\langle i|0 \rangle$ for $i \in [0, k-1]$.*

PROOF OF LEMMA 6.2. The forwards implication is straightforward, as discussed above. For the backwards implication, note that if the ket $|j \rangle$ is only present in brackets $\langle j-1 \bmod k|j \rangle$, then there are at least as many agents with bra $\langle j-1 \bmod k|$ as agents with bra $\langle j|$. Therefore $x_{j-1} > x_j$ for all $j \in [1, k-1]$, and the result holds. $\square$

Thus the idea is to recycle the CIRCLES protocol to be in the situation where the input is the initial order of the colors. For that, we now introduce a $O(k^4)$ protocol where the bra and ket consist now of 2 different values: $\langle i, i|$ and $|i, i \rangle$ for every $i \in [0, k-1]$. $i \in [0, k-1]$ represents the (belief of) ranking of the color i . Now, when applying the CIRCLES protocol, the idea is that instead of computing the weights according to the colors value, we compute them according to the rankings' value. Formally, for every $i, j \in [0, k-1]$

$$w(\langle i, i|j, j \rangle) = \begin{cases} k & \text{if } i = j \\ (j - i) \bmod k & \text{otherwise} \end{cases} \quad (1)$$

It remains to discuss how the value i is assigned. We start by assigning $i \leftarrow i$. Moreover, every agent is assigned a token which makes it a leader for its color, that is, this agent is in charge of assigning the ranking of the color to the corresponding color, and broadcast any changes that it decides. When two tokens for the same color meet, one of them has to be deleted. When two tokens for different colors $i < j$ but same ranking $i = j$ meet, one of them increases its ranking by one $j \leftarrow j + 1$. Eventually this ensures that there is a unique leader for each color, and all the rankings are different.

Finally, rules should be implemented to ensure that whenever the ranking assigned is not correct, agents are able to change their rankings. We note that if the rankings are not correct, but are unique, and the CIRCLES protocol has stabilized, then there exists two agents of the form $\langle i, i|j, j \rangle$ and $\langle i, i|l, l \rangle$ with $j \neq l$ and $j, l \neq 0$, or there must exist an agent $\langle i, i|i, i \rangle$ with $i \neq 0$. We now give the lemma that hints at how agents can determine that the current ranking is wrong.

LEMMA 6.3. *In the CIRCLES protocol, after stabilization, if the initial colors do not satisfy $x_0 > x_1 > \dots > x_{k-1}$, and $x_i \neq 0 \ \forall i$, then either:*

- (1) *There exist an agent $\langle i|i \rangle$ with $i \neq 0$, or*
- (2) *there exist two agents of the form $\langle i|j \rangle$ and $\langle i|l \rangle$ with $j \neq l$ and $j, l \neq 0$.*

PROOF. Assume all agents of the form $\langle i|i \rangle$ satisfy $i = 0$, that is, 0 is the majority color. And assume that there exists a j such that $x_j < x_{j+1}$, and let ℓ be the smallest such j . In particular, $\ell \neq 0$ and $x_{\ell-1} > x_\ell$. Therefore, all of the kets $|l \rangle$ are in agents of the form $\langle \ell - 1|l \rangle$, and at least one of such agents exists since $x_\ell \neq 0$. Moreover, the first greedy independent set that does not contain ℓ contains both $\ell - 1$ and $\ell + 1$. Therefore there exists an agent of the form $\langle \ell - 1|\ell + 1 \rangle$. $\square$

In our case, this means that whenever two agents of the form $\langle i, i|j, j \rangle$ and $\langle i, i|l, l \rangle$ with $j \neq l$ and $j, l \neq 0$ meet, they need to exchange j and l . This can only be allowed if they respectively hold the tokens for j and l . Moreover, $\langle i, i|i, i \rangle$ must be able to claim the ranking $i = 0$. We therefore enforce the token of color j to be attached to a ket $|j, j \rangle$. The token should seek the agent $\langle i, i|j, j \rangle$ with minimal i to stick to, that is, whenever $\langle i, i|j, j \rangle^*$ meets $\langle \ell, l|j, j \rangle$ (where the asterisk denotes the token on the color of the ket), the token should be transferred to $\langle \ell, l|j, j \rangle$ only if $l < i$. We can then enforce that whenever $\langle i, i|i, i \rangle^*$ meets $\langle j, j|0, 0 \rangle^*$, they exchange their rankings, that is $l \leftarrow i, i \leftarrow 0$; and when $\langle i, i|j, j \rangle^*$ and $\langle i, i|l, l \rangle^*$ meet, they also exchange their rankings, that is $j, l \leftarrow i, i$.

With all of these rules, we are done describing the protocol, and it remains to show that it is correct against a globally fair scheduler. For that, we show below that there is only one configuration that is stable, namely the one where all the assigned rankings of the colors is correct, and where the CIRCLES protocol is stabilized. We then show that this configuration is reachable from all other valid

configurations (that is, configurations reachable from the initial configuration), and this is enough to conclude.

Before that, let us describe more formally the protocol. In this description, whenever two agents meet, if multiple cases apply, select only one case: the first case that does not leave the agents unchanged. The $\circ$ symbol denotes the fact that the existence or not of a token is not important:

- (1) Initialization:
 - Each agent in color i starts in the state $\langle i, i = i|i = i, i \rangle^*$
- (2) Token deletion:
 - If $\langle j, j|i, i \rangle^*$ meets $\langle \ell, l|i', i' \rangle^*$, with $j \leq l$, then $\langle \ell, l|i', i' \rangle^*$ becomes $\langle \ell, l|i, i \rangle$
- (3) Ranking uniqueness:
 - If $\langle j, j|i, i \rangle^*$ meets $\langle \ell, l|u, u \rangle^*$, with $i = u$ and $i < u$, then $\langle \ell, l|u, u \rangle^*$ becomes $\langle \ell, l|u + 1, u \rangle^*$ (in other words, $u \leftarrow u + 1$).
- (4) Ranking broadcast:
 - If $\langle i, i|j, j \rangle^*$ meets $\langle u, u|j', j' \rangle$, then $\langle u, u|j', j' \rangle$ becomes $\langle u, u|j, j \rangle$.
 - If $\langle i, i|j, j \rangle^*$ meets $\langle j, j'|u, u \rangle^\circ$, then $\langle j, j'|u, u \rangle^\circ$ becomes $\langle j, j|u, u \rangle^\circ$
- (5) CIRCLES:
 - Recall the weight function from Equation (1).
 - If agent $\langle i, i|j, j \rangle^\circ$ meets $\langle u, u|v, v \rangle^\circ$ with $\min\{w(\langle i, i|v, v \rangle), w(\langle u, u|j, j \rangle)\} < \min\{w(\langle i, i|j, j \rangle), w(\langle u, u|v, v \rangle)\}$, then $\langle i, i|j, j \rangle^\circ$ becomes $\langle i, i|v, v \rangle^\circ$ and $\langle u, u|v, v \rangle^\circ$ becomes $\langle u, u|j, j \rangle^\circ$ (If there are any tokens, they are exchanged to stay with their respective kets).
- (6) Token transfer:
 - If $\langle i, i|i, i \rangle^*$ meets $\langle \ell, l|i, i \rangle^*$, then $\langle \ell, l|i, i \rangle^*$ becomes $\langle \ell, l|i, i \rangle$ and $\langle i, i|i, i \rangle^*$ becomes $\langle i, i|i, i \rangle^*$
 - If $\langle j, j|i, i \rangle^*$ meets $\langle \ell, l|i, i \rangle^*$, with $j \neq i, \ell \neq i$ and $j < l$, then $\langle \ell, l|i, i \rangle^*$ becomes $\langle \ell, l|i, i \rangle$ and $\langle j, j|i, i \rangle^*$ becomes $\langle j, j|i, i \rangle^*$
- (7) Ranking swap:
 - If $\langle i, i|i, i \rangle^*$ meets $\langle u, u|0, v \rangle^*$ with $v \neq i, i \neq 0$, then $\langle i, i|i, i \rangle^*$ becomes $\langle i, i|0, i \rangle^*$ and $\langle u, u|0, v \rangle^*$ becomes $\langle u, u|i, v \rangle^*$
 - If $\langle i, i|j, j \rangle^*$ meets $\langle i, i|l, l \rangle^*$ with $j, l \neq i$, then $\langle i, i|j, j \rangle^*$ becomes $\langle i, i|l, j \rangle^*$ and $\langle i, i|l, l \rangle^*$ becomes $\langle i, i|j, l \rangle^*$.
- (8) Output:
 - An agent in state $\langle i, i|j, j \rangle$ outputs i .

We now define the configuration that we show below is stable:

Definition 6.4. Let X be the following configuration:

- (1) There exists exactly k tokens, one for each $i \in [0, k - 1]$. Each token is on a ket of different color.
- (2) The tokens are associated to k different rankings, defined by: $i \in [0, k - 1], r(i) := |\{\ell \in [0, k - 1] : x_\ell > x_i\}|$.
- (3) Every bra and ket are of the form $\langle i, r(i) \rangle$ for $i \in [0, k - 1]$ or $|r(j), j \rangle$ for $j \in [0, k - 1]$
- (4) The CIRCLES protocol is stabilized, that is, all kets are of the form either $\langle i, r(i)|r(j) = r(i) + 1, j \rangle^\circ$ or $\langle i, r(i)|r(j) = 0, j \rangle^\circ$
- (5) All tokens are on brackets of the form $\langle i, r(i) = 0|r(i) = 0, i \rangle^*$ or $\langle i, r(i)|r(j) = r(i) + 1, j \rangle^*$.

LEMMA 6.5 (★). X is a stable configuration.

We finally show that X is reachable from every configuration.

PROPOSITION 6.6. *X is reachable from any valid configuration.*

The rest of this section is dedicated to proving this proposition. We do so by taking any valid configuration, and giving a series of intermediate configurations, each reachable from the previous configuration, until we reach X . Let A be any valid configuration.

LEMMA 6.7. *A satisfies the following conditions:*

- (1) *for every i , there exists exactly x_i many bras and x_i many kets of the form $\langle i, \cdot |$ and $|\cdot, i \rangle$ respectively.*
- (2) *for every i , there exist at least one ket of the form $|\cdot, i \rangle^*$*

PROOF. Both statements are straightforward, as the first and second elements of the bras and kets respectively are never changed, only exchanged, and as the token is passed around in a leader election fashion over the kets of the form $|\cdot, i \rangle$. $\square$

LEMMA 6.8. *There exists a configuration B reachable from A with the following conditions:*

- (1) *for every i , there exists exactly x_i many bras and x_i many kets of the form $\langle i, \cdot |$ and $|\cdot, i \rangle$ respectively.*
- (2) *for every i , there exist **exactly** one ket of the form $|\cdot, i \rangle^*$.*

PROOF. Again, this is straightforward using Rules (2) and (3), and the fact that the second elements of kets are only exchanged. $\square$

LEMMA 6.9. *There exists a configuration C reachable from B with the following conditions:*

- (1) *for every i , there exists exactly x_i many bras and x_i many kets of the form $\langle i, \cdot |$ and $|\cdot, i \rangle$ respectively.*
- (2) *for every i , there exist exactly one ket of the form $|\cdot, i \rangle^*$. Moreover, the pairs formed by these kets form a bijection of $[0, k-1]$ to $[0, k-1]$, that is to say, the rankings of the tokens are unique.*

PROOF. From configuration B , we can select the pairs of agents to interact that only use Rule (3) to get the desired configuration. $\square$

LEMMA 6.10. *There exists a configuration D reachable from C with the following conditions:*

- (1) *for every i , there exists exactly x_i many bras and x_i many kets of the form $\langle i, \cdot |$ and $|\cdot, i \rangle$ respectively.*
- (2) *for every i , there exist exactly one ket of the form $|\cdot, i \rangle^*$. Moreover, the pairs formed by these kets form a bijection of $[0, k-1]$ to $[0, k-1]$, that is to say, the rankings of the tokens are unique.*
- (3) *The rankings of all bras and kets are consistent with the rankings assigned by the tokens.*

PROOF. From configuration C , rules (2) and (3) do not change any agent anymore. Therefore, from C , it suffices for each agent with a token to meet the agents with the corresponding bra or ket to assign them the correct ranking. $\square$

LEMMA 6.11. *There exists a configuration E_{-1} reachable from D with the following conditions:*

- (1) *for every i , there exists exactly x_i many bras and x_i many kets of the form $\langle i, \cdot |$ and $|\cdot, i \rangle$ respectively.*
- (2) *for every i , there exist exactly one ket of the form $|\cdot, i \rangle^*$. Moreover, the pairs formed by these kets form a bijection of $[0, k-1]$ to $[0, k-1]$, that is to say, the rankings of the tokens are unique.*

- (3) *The rankings of all bras and kets are consistent with the rankings assigned by the tokens.*
- (4) *The CIRCLES protocol is stabilized on this particular ranking, that is, Rule (5) cannot modify anymore any pair of agents.*

PROOF. From configuration D , Rules (2) to (4) cannot modify any agent anymore. Since the CIRCLES protocol works against a weakly fair scheduler, there exists a sequence of interactions that lead to its stabilization. Following this sequence, by skipping any interaction that wouldn't modify any agents, we reach E_{-1} . $\square$

LEMMA 6.12 (★). *For every $\ell \in [0, k-1]$, there exists a configuration E_ℓ reachable from $E_{\ell-1}$ with the following conditions:*

- (1) *for every i , there exists exactly x_i many bras and x_i many kets of the form $\langle i, \cdot |$ and $|\cdot, i \rangle$ respectively.*
- (2) *for every i , there exist exactly one ket of the form $|\cdot, i \rangle^*$. Moreover, the pairs formed by these kets form a bijection of $[0, k-1]$ to $[0, k-1]$, that is to say, the rankings of the tokens are unique.*
- (3) *The rankings of all bras and kets are consistent with the rankings assigned by the tokens.*
- (4) *The CIRCLES protocol is stabilized on this particular ranking, that is, Rule (5) cannot modify anymore any pair of agents.*
- (5) *E_ℓ contains the kets $|\mathbf{r}(j), j \rangle^*$ for all $i \leq \ell$, that is, all rankings up to ℓ are correct.*

As $E_{k-1} = X$, this concludes the proof of Proposition 6.6.

PROOF OF THEOREM 1.2. Since X is a stable configuration, by Lemma 6.5, and is reachable from any other configuration by Proposition 6.6, we know that X is the only stable configuration, and is always reached under a globally fair scheduler. $\square$

7 Future work

Our protocol solves relative majority always-correctly with $O(k^3)$ states, narrowing the gap with the $\Omega(k^2)$ lower bound. Its versatility further allows computing the full ranking, making it a promising building block for more complex distributed tasks. During our work, both CIRCLES and an independent protocol based on a different logic hit $\Omega(k^3)$ states as a hard barrier. We therefore believe the most fruitful direction is strengthening the $\Omega(k^2)$ lower bound. Such improvements are rare in the population protocol model, and we hope this work motivates the development of new lower bound techniques with broader impact on population protocols.

Acknowledgments

Funded by the European union. Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or the European Research Council Executive Agency. Neither the European Union nor the granting authority can be held responsible for them. This project has received funding from the European Research Council (ERC) under the European Union's Horizon 2020 research and innovation programme (MoDynStruct, No. 101019564)  and the Austrian Science Fund (FWF) grant DOI 10.55776/I5982. For open access purposes, the author has applied a CC BY public copyright license to any author-accepted manuscript version arising from this submission.

References

- [1] Dan Alistarh, James Aspnes, David Eisenstat, Rati Gelashvili, and Ronald L. Rivest. 2017. Time-Space Trade-offs in Population Protocols. In *Proceedings of the 28th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA 2017)*, Philip N. Klein (Ed.). SIAM, Barcelona, Spain, 2560–2579. doi:10.1137/1.9781611974782.169
- [2] Dan Alistarh, James Aspnes, and Rati Gelashvili. 2018. Space-Optimal Majority in Population Protocols. In *Proceedings of the 29th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA 2018)*, Artur Czumaj (Ed.). SIAM, New Orleans, LA, USA, 2221–2239. doi:10.1137/1.9781611975031.144
- [3] Dan Alistarh, Krishnendu Chatterjee, Mehrdad Karrabi, and John Lazarsfeld. 2024. Game Dynamics and Equilibrium Computation in the Population Protocol Model. In *Proceedings of the 43rd ACM Symposium on Principles of Distributed Computing (PODC 2024)*, ACM, New York, NY, USA, 40–49. doi:10.1145/3662158.3662768
- [4] Dan Alistarh and Rati Gelashvili. 2018. Recent Algorithmic Advances in Population Protocols. *ACM SIGACT News* 49, 3 (2018), 63–73. doi:10.1145/3289137.3289150
- [5] Dan Alistarh, Rati Gelashvili, and Milan Vojnović. 2015. Fast and Exact Majority in Population Protocols. In *Proceedings of the 2015 ACM Symposium on Principles of Distributed Computing (PODC 2015)*, ACM, New York, NY, USA, 47–56. <https://doi.org/10.1145/2767386.2767429>
- [6] Talley Amir, James Aspnes, Petra Berenbrink, Felix Biermeier, Christopher Hahn, Dominik Kaaser, and John Lazarsfeld. 2023. Fast Convergence of k-Opinion Undecided State Dynamics in the Population Protocol Model. In *Proceedings of the 2023 ACM Symposium on Principles of Distributed Computing (PODC 2023)*, ACM, Orlando, FL, USA, 13–23. doi:10.1145/3583668.3594589
- [7] Dana Angluin, James Aspnes, Zoë Diamadi, Michael J. Fischer, and René Peralta. 2006. Computation in networks of passively mobile finite-state sensors. *Distributed computing* 18, 4 (2006), 235–253. doi:10.1145/1011767.1011810
- [8] James Aspnes, Joffroy Beauquier, Janna Burman, and Devan Sohler. 2016. Time and Space Optimal Counting in Population Protocols. In *Proceedings of the 20th International Conference on Principles of Distributed Systems (OPODIS 2016) (LIPIcs)*, Schloss Dagstuhl - Leibniz-Zentrum für Informatik, Madrid, Spain, 13:1–13:17. doi:10.4230/LIPICS.OPODIS.2016.13
- [9] Gregor Bankhamer, Petra Berenbrink, Felix Biermeier, Robert Elsässer, Hamed Hosseinpour, Dominik Kaaser, and Peter Kling. 2022. Fast Consensus via the Unconstrained Undecided State Dynamics. In *Proceedings of the 2022 ACM-SIAM Symposium on Discrete Algorithms, (SODA 2022)*, SIAM, Alexandria, VA, USA, 3417–3429. doi:10.1137/1.9781611977073.135
- [10] Gregor Bankhamer, Petra Berenbrink, Felix Biermeier, Robert Elsässer, Hamed Hosseinpour, Dominik Kaaser, and Peter Kling. 2022. Population Protocols for Exact Plurality Consensus: How a small chance of failure helps to eliminate insignificant opinions. In *Proceedings of the 2022 ACM Symposium on Principles of Distributed Computing (PODC 2022)*, ACM, Salerno, Italy, 224–234. doi:10.1145/3519270.3538447
- [11] Luca Becchetti, Andrea E. F. Clementi, Emanuele Natale, Francesco Pasquale, and Riccardo Silvestri. 2015. Plurality consensus in the gossip model. In *26th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA 2015)*, ACM, San Diego, USA, 371–390. doi:10.1137/1.9781611973730.27
- [12] Petra Berenbrink, Robert Elsässer, Tom Friedetzky, Dominik Kaaser, Peter Kling, and Tomasz Radzik. 2021. Time-space trade-offs in population protocols for the majority problem. *Distributed Comput.* 34, 2 (2021), 91–111. doi:10.1007/S00446-020-00385-0
- [13] Petra Berenbrink, Tom Friedetzky, George Giakkoupis, and Peter Kling. 2016. Efficient Plurality Consensus, Or: the Benefits of Cleaning up from Time to Time. In *Proceedings of the 43rd International Colloquium on Automata, Languages, and Programming (ICALP 2016)*, Rome (Italy), 136:1–136:14. doi:10.4230/LIPICS.ICALP.2016.136
- [14] Michael Blondin and François Ladouceur. 2023. Population Protocols with Unordered Data. In *Proceedings of the 50th International Colloquium on Automata, Languages, and Programming, (ICALP 2023) (LIPIcs, Vol. 261)*, Schloss Dagstuhl - Leibniz-Zentrum für Informatik, Paderborn, Germany, 115:1–115:20. doi:10.4230/LIPICS.ICALP.2023.115
- [15] James M. Bower and Hamid Bolouri (Eds.). 2001. *Computational Modeling of Genetic and Biochemical Networks*. The MIT Press. doi:10.7551/mitpress/2018.001.0001
- [16] Shukai Cai, Taisuke Izumi, and Koichi Wada. 2012. How to Prove Impossibility Under Global Fairness: On Space Complexity of Self-Stabilizing Leader Election on a Population Protocol Model. *Theory Comput. Syst.* 50 (04 2012), 433–445. doi:10.1007/s00224-011-9313-z
- [17] Luca Cardelli and Attila Csikász-Nagy. 2012. The cell cycle switch computes approximate majority. *Scientific reports* 2, 1 (2012), 656–656.
- [18] Philipp Czermer, Javier Esparza, and Jérôme Leroux. 2023. Lower bounds on the state complexity of population protocols. *Distributed computing* 36, 3 (2023), 209–218. doi:10.1007/S00446-023-00450-4
- [19] David Doty. 2014. Timing in chemical reaction networks. In *25th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA 2014)*, Portland (USA), 772–784. doi:10.1137/1.9781611973402.57
- [20] David Doty, Mahsa Eftekhari, Leszek Gasieniec, Eric E. Severson, Przemysław Uznanski, and Grzegorz Stachowiak. 2021. A time and space optimal stable population protocol solving exact majority. In *62nd IEEE Annual Symposium on Foundations of Computer Science, (FOCS 2021)*, IEEE, Denver, CO, USA, 1044–1055. doi:10.1109/FOCS52979.2021.00104
- [21] Moez Draief and Milan Vojnović. 2012. Convergence Speed of Binary Interval Consensus. *SIAM Journal on Control and Optimization* 50, 3 (2012), 1087–1109. doi:10.1137/110823018
- [22] Antoine El-Hayek, Robert Elsässer, and Stefan Schmid. 2025. An Almost Tight Lower Bound for Plurality Consensus with Undecided State Dynamics in the Population Protocol Model. In *Proceedings of the ACM Symposium on Principles of Distributed Computing, (PODC 2025)*, ACM, Huatulco, Mexico, 532–540. doi:10.1145/3732772.3733505
- [23] Robert Elsässer and Tomasz Radzik. 2018. Recent Results in Population Protocols for Exact Majority and Leader Election. In *The Distributed Computing Column*, Stefan Schmid (Ed.).
- [24] Leszek Gasieniec, David Hamilton, Russell Martin, Paul G. Spirakis, and Grzegorz Stachowiak. 2017. Deterministic Population Protocols for Exact Majority and Plurality. In *Proceedings of the 20th International Conference on Principles of Distributed Systems (OPODIS 2016)*, Madrid (Spain), 14:1–14:14. doi:10.4230/LIPICS.OPODIS.2016.14
- [25] George B. Mertzios, Sotiris E. Nikolettseas, Christoforos L. Raptopoulos, and Paul G. Spirakis. 2017. Determining majority in networks with local interactions and very small local memory. *Distributed Comput.* 30, 1 (2017), 1–16. doi:10.1007/S00446-016-0277-8
- [26] Emanuele Natale and Iliad Ramezani. 2019. On the Necessary Memory to Compute the Plurality in Multi-Agent Systems. In *Proceedings of the 11th International Conference on Algorithms and Complexity (CIAC 2019)*, Rome (Italy), 323–338. <http://arxiv.org/abs/1901.06549>
- [27] Saber Salehkaleybar, Arsalan Sharif-Nassab, and S. Jamaloddin Golestani. 2015. Distributed Voting/Ranking With Optimal Number of States per Node. *IEEE transactions on signal and information processing over networks* 1, 4 (2015), 259–267. doi:10.1109/TSIPN.2015.2477777

8 Missing Proof Details

8.1 Proof of Lemma 3.2

PROOF. Each time a set G_p is constructed (following the construction process in Definition 3.1), any color whose count is not zero yet is added into G_p and its count is decremented by one. The color μ appears in the population strictly more than any other, it therefore holds that $\forall p \in [1, q], \mu \in G_p$. Let now $i \in [0, k - 1]$ be a color contained in a set G_p for some $p \in [1, q]$. It holds that $\forall l \leq p, i \in G_l$ as because color i is available to populated G_p , it was also available when G_l was filled earlier. It is therefore not possible that a color $j \neq \mu$ is contained in G_q as j would be contained in all sets and thus j would also be in relative majority, contradicting μ being the unique color in relative majority. $\square$

8.2 Proof of Lemma 3.3

PROOF. Every agent is initialized with bracket $\langle i | i \rangle$ for some $i \in [0, k - 1]$ and the claim initially holds. Agents subsequently only ever update their bracket, by exchanging kets among each other. The overall number of bras and kets in the population therefore does not change during a computation. $\square$

8.3 Proof of Theorem 4.1

THEOREM 4.1 (★). *CIRCLES_TIEREPORT solves the relative majority problem, where all agents output the color of the majority if the majority is strict. If there is a tie, agents do not output a color, but rather report that there is a tie. It requires $3k^3$ many states, which can be reduced to $4k^2$ if agents are not required to output the majority color, but only whether their color wins or loses.*

The result follows from Theorem 3.5 together with the following lemma.

LEMMA 8.1. *If there is no tie in the input colors then all agents eventually are at `no_tie`. If there is a tie in the input colors then all agents eventually are at `tie` or `tie_spreader`.*

PROOF. We first assume that there is no tie. The only rules that sets an agent to `tie_spreader` is the first rule; this rule eventually stops triggering since all agents stop exchanging their ket past a certain time (Theorem 3.4). As there is no tie, there will be a remaining agent in state $\langle \mu | \mu \rangle$ past a certain time and that agent eventually sets all other agents to `no_tie`.

We now assume that there is a tie. Past a certain time, no agent will be in state $\langle i | i \rangle$ for any i . There hence exists an interaction where the last agent of the form $\langle i | i \rangle$ decays into $\langle i | j \rangle$ with $i \neq j$. When this happens that agent $\langle i | j \rangle$ is at `tie_spreader` and will set all other agents at `no_tie` to `tie`. $\square$

8.4 Proof of Theorem 4.2

THEOREM 4.2 (★). *Let $M = \mu_0, \mu_1, \dots, \mu_{r-1}$ be the colors in relative majority. In the `CIRCLES_TIESHARING` protocol, eventually all agents with initial color in M are at `winner_spreader` or `winner` while all other agents are at `loser`. Moreover, the output of every winner or `winner_spreader` is its initial color, while the output of loser agents is one of the winning colors.*

The protocol uses $3k^3$ many states. This can be dropped to $3k^2$ if we only require each agent to only output whether their color win or lose.

To prove this theorem, we split it into two cases, depending on whether $r = 1$ or $r > 1$. In the rest of this section, we simplify the notation and write μ_s for $\mu_{s \bmod r}$.

LEMMA 8.2. *If $r = 1$, all agents with initial color in M are at `winner` while all other agents are at `loser`.*

PROOF. Eventually the only agents with brackets $\langle i | i \rangle$ are $\langle \mu_0 | \mu_0 \rangle$, by Lemmas 3.7 and 3.2. Those agents are at `winner` from either the initial configuration or the `token destructor` rule. Moreover, those agents will set all agents $\langle \mu_0 | j \rangle$ to `winner` and all agents $\langle i | j \rangle$ with $i \neq \mu_0$ to `loser`, by the `token destruction` rule. The out of all agents is set to μ_0 by rule (i). $\square$

The idea for the case $r > 1$ relies heavily on the following observation:

CLAIM 2. *After Stabilization (Theorem 3.4), every bracket $\langle i | j \rangle$ held by an agent satisfies that there exists an s such that $[i, j]_k \subseteq [\mu_s, \mu_{s+1}]_k$.*

This claim is immediate by the definitions of the Greedy independent sets (Definition 3.1), the corresponding Circle Bracket Sets (Definition 3.6) and Lemma 3.7.

We thus show below that starting at some point in time, there exists a token in every interval $[\mu_s, \mu_{s+1}]_k$, there exists an agent $\langle \mu_s | \mu_{s+1} \rangle$ that holds a token, and can spread the information for all agents in that interval.

We start by proving the following lemma. It relies on our definition of Greedy independent sets (Definition 3.1) and the corresponding Circle Bracket Sets (Definition 3.6).

LEMMA 8.3. *Let $r > 1$. Let $T(t)$ be the set of agents that have a token after interaction $t \in \mathbb{N}$. There exists an $\ell \in \mathbb{N}$ and a set of agents*

T such that for all $t \geq \ell$, $T(t) = T$. Moreover, the multiset of brackets of all agents in T is equal to $f(G_q)$.

PROOF. Let t be the interaction where the brackets of the agents stop changing (Theorem 3.5). The population contains agents with brackets $\langle \mu_0 | \mu_1 \rangle, \langle \mu_1 | \mu_2 \rangle, \dots, \langle \mu_{r-1} | \mu_0 \rangle$, by Lemma 3.7.

We show that starting at ℓ , we have the following invariant:

CLAIM 3. *For any $t' \geq \ell$, we have that for any $s \in [1, r]$, there exists a $v \in [0, k-1]$ with $[\mu_s, v]_k \subseteq [\mu_s, \mu_{s+1}]_k$, and such that there exists an agent with bracket $\langle \mu_s | v \rangle$ either at `winner_spreader` or `loser`.*

PROOF. Let us fix an s and look at the last point in time τ where a $\langle \mu_s |$ exchanged kets. By `token generation`, a token is thus created. We then prove the claim by induction. Assume that after interaction $\tau' \geq \tau$, we have that there exists a $v \in [0, k-1]$ with $[\mu_s, v]_k \subseteq [\mu_s, \mu_{s+1}]_k$, and such that there exists an agent with bracket $\langle \mu_s | v \rangle$ either at `winner_spreader` or `loser`. We prove this holds as well after interaction $\tau' + 1$.

If the interaction did not involve a bracket $\langle \mu_s | v \rangle$ at either `loser` or `winner_spreader`, the induction is immediate. We thus assume that such an agent is involved in the interaction.

If the interaction $\tau' + 1$ is either `token generation`, or `token destructor`, the results holds trivially, as that same agent is set to either `winner_spreader` or `loser` (It cannot be set at `winner` using `token destructor`, as that would imply the existence of a $\langle \mu_s | \mu_s \rangle$, which is incompatible with the fact that $r > 1$ and the definition of τ).

If interaction $\tau' + 1$ is a `token destruction`, agent in $\langle \mu_s | v \rangle$ must have been set to `loser`, as the alternative (being set to `winner` would mean that $\langle \mu_s | v \rangle$ met $\langle \mu_s | \mu_s \rangle$, a contradiction to τ being the last point in time where $\langle \mu_s |$ exchanged kets, as we know by Lemma 3.7 that $\langle \mu_s | \mu_s \rangle$ is not in the final bracket multiset). This maintains the invariant.

If interaction $\tau' + 1$ is a `token ascent`, then either agent in $\langle \mu_s | v \rangle$ must have been set to `loser`, or the agent it interacted with had a token and was in a state of the form $\langle \mu_s | j \rangle$ for some $j \in [0, k-1]$. In that case the invariant holds after $\tau' + 1$, because either $\langle \mu_s | v \rangle$ is at `loser`, or $\langle \mu_s | j \rangle$ being at `winner_spreading`.

If interaction $\tau' + 1$ is a `token simplification`, then either agent in $\langle \mu_s | v \rangle$ must have been set to `loser`, or the agent it interacted with had a token and was in a state of the form $\langle \mu_s | j \rangle$ for some $j \in [0, k-1]$. In that case the invariant holds after $\tau' + 1$, because either $\langle \mu_s | v \rangle$ is at `loser`, or $\langle \mu_s | j \rangle$ being at `winner_spreading`.

If interaction $\tau' + 1$ is an `information spreading`, either $\langle \mu_s | v \rangle$ is set to `loser`, which maintains the invariant, or it met some bracket $\langle \mu_s | j \rangle$ for some $j \in [0, k-1]$ at `winner_spreader`. In that case the invariant holds after $\tau' + 1$, not necessarily due to $\langle \mu_s | v \rangle$, but rather $\langle \mu_s | j \rangle$ being at `winner_spreading`. $\square$

At time t , the population also contains at least one agent at `winner_spreader`, which was created by the last ket exchange. This token will ascend until it becomes attached to a $\langle \mu_s | \mu_{s+1} \rangle$ for some $s \in [0, k-1]$. In case there is no token associated to a $\langle \mu_{s+1} | j \rangle$, by Claim 3, there is at least one agent in $\langle \mu_{s+1} | j \rangle$ at `loser`, and the $\langle \mu_s | \mu_{s+1} \rangle$ agent at `winner_spreader` will generate a token at it by `token multiplication`, which will then ascend as well. We can thus show inductively that after some point, for all s , there exists a token in one of the $\langle \mu_s | \mu_{s+1} \rangle$, using Claim 3, `token multiplication` and `token`

ascent. (Note that *token destruction* cannot happen anymore as we are after Stabilization (Theorem 3.4), while *token simplification* ensures at least one token remains. *Information spreading* will then ensure that all agents in $\langle \mu_s | j \rangle$ for some $s \in [1, r]$, $j \in [0, k-1]$ are set to *winner*, blocking any further *token multiplication* to happen. *Token simplification* then ensures that there is exactly one token associated to a $\langle \mu_s | \mu_{s+1} \rangle$ for each s . $\square$

PROOF OF THEOREM 4.2. The case for $r = 1$ is handled by Lemma 8.2. The case for $r \geq 2$ is mostly handled by Lemma 8.3, where we conclude by noting that agents in T set all other agents to either *winner* or *loser* correctly using *information spreading*, and update their *out* accordingly (the key point here is that after Stabilization (Theorem 3.4), every bracket $\langle i | j \rangle$ satisfies that there exists an s such that $[i, j]_k \subseteq [\mu_s, \mu_{s+1}]_k$).

The claim on the number of states is straightforward, as each agent stores a bracket that can have k^2 values, a status that can have three values, and eventually an output with k possible values. $\square$

8.5 Proof of Theorem 4.3

THEOREM 4.3 (★). *CIRCLES_TIEBREAK solves the relative majority problem against a weakly-fair scheduler. If there is a tie, agents all output the same color, one of the majority ones.*

It requires $2k^3$ many states, which can be reduced to $3k^2$ if agents are not required to output the majority color, but only whether their color wins or loses.

The protocol stores three numbers between 0 and $k-1$ and the *tbt* bit, giving $2k^3$ possible states. It is possible to remove the memory entry *out* and add a single bit, which stores only whether and agent is in relative majority or not. For that an agent holding the tie-break token sets this bit to true for itself and all agents it meets with the same bra and to false for all other agents. This requires $3k^2$ states (because any agent holding a tie-break token has the output bit to 1). The overall claim follows from the following lemma.

LEMMA 8.4. *After some number of interactions all agents store the same color in out, which is part of a (the) color in relative majority.*

PROOF. By Theorem 3.4 after some number of interactions t_1 , the bracket of every agent remains unchanged. In the corner case that $t_1 = 0$, meaning agents never exchange kets, only a single color is present in the population. In this case no tie-break token is ever generated and *out* of all agents contains the unique color by initialization, producing the correct output.

Otherwise in t_1 the last exchange of kets between agents occurs. Therefore a tie-break token is generated in t_1 . This means at least one such token exists after t_1 . It is, however, possible for multiple tokens to exist at this point, generated during previous interactions. After t_1 no more tie-break tokens are generated, because step (1) is never executed again. Step (2) destroys one token whenever two tokens meet, decreasing the overall number of tokens. Since, however, two tokens are required for it to be executed, the number of tokens can never be decreased from 1 to 0, meaning at least one token exists in every computation step after t_1 . After t_1 the weight of agents does not change anymore. The token transfer in combination with the destruction step then ensures that all tokens are eventually passed to the agent with maximum weight and among them to the one with the largest bra. If multiple agents with maximum weight

and maximum bra exist, then the destruction step still ensures that eventually only one of them holds the tie-break token. This means after some number $t_2 \geq t_1$ of interactions, among all agents of maximum weight a unique agent a exists that holds the tie-break token. Due to the output propagation, after some number $t_3 \geq t_2$ of interactions, all agents store $\langle i_a | \rangle$ as output.

All that is left to show is that $\langle i_a | \rangle$ represents a color in relative majority. Associate with the population sets $G_1, G_2, \dots, G_q$ as in Definition 3.1. By Lemma 3.7 after t_1 the population consists of agents with brackets $\bigcup_{p \in [q]} f(G_p)$. Clearly G_q contains all colors in relative majority. In case $f(G_q)$ contains a single element, then this element has the largest possible weight k . Otherwise, the maximum weight is obtained in $f(G_q)$ because $G_1 \supseteq G_2 \supseteq \dots \supseteq G_q$. Therefore $\langle i_a | \rangle$ represents a color in relative majority, which concludes the proof. $\square$

8.6 Proof of Theorem 5.1

THEOREM 5.1 (★). *ORDERING solves the ordering problem with $O(k^2)$ states.*

Formally we define the protocol as follows.

Memory organization: The state of an agent a encodes the following information: (1) ld_a , a single bit indicating whether a is a leader (2) ic_a the initial color of a (in an arbitrary representation) (3) d_a the label of ic_a in binary representation using $\lceil \log_2 k \rceil$ bits

Initialization: Initially every agents sets $ld = 1$, $d = 0$, and ic is initialized with the agents initial color.

Transition function: The transition function that updates the states of two agents $a, b \in A$ is given by the following pseudocode.

```

1: if  $ic_a = ic_b$  then
2:   if  $ld_a = 1$  and  $ld_b = 1$  then
3:      $ld_a \leftarrow 0$ 
4:   if  $ld_a = 0$  and  $ld_b = 1$  and  $d_a \neq d_b$  then
5:      $d_a \leftarrow d_b$ 
6: else
7:   if  $d_a = d_b$  and  $ld_a = 1$  and  $ld_b = 1$  then
8:      $d_a \leftarrow (d_a + 1) \bmod k$ 

```

Output: The output of agent a can be read directly from d_a .

We show different properties of the protocol, that we will then use to argue its correctness. For any color $c \in C$, let $\text{supp}(c)$ denote the set of agents initially supporting color c . First we consider the leader election process.

LEMMA 8.5. *After some number of interactions t_1 each non-empty set $\text{supp}(c)$ of agents contains exactly one leader. After t_1 the leader bit of all agents will not change.*

PROOF. The proof is highly similar to that of Lemma 1 in [26]. Some additional argumentation similar to that of proof of Lemma 7 in [16] is added. At no point does ORDERING change a leader bit from 0 to 1. The number of leaders can subsequently never increase. Let $\#_c$ denote the number of leaders with color c for some $c \in C$. When two leaders of color c interact, one of them is set to be a non-leader, due to line 3 of the transition function, decreasing $\#_c$ by exactly one. As long as there are at least two leaders of color

c , $\#_c$ will be decremented in that way. Two agents are required to clear a leader bit and reduce $\#_c$. Therefore no interaction reduces $\#_c$ from 1 to 0 as soon as there is only a single agent left. Therefore there is exactly one leader for color c , whose leader bit will never be set to zero. This holds for any $c \in C$, meaning there is exactly one leader per color.

No leader bit of a leader can be set to 0 after that point and no leader bit of a non-leader can be set to 1. Therefore no leader bit changes anymore. $\square$

A configuration containing a unique leader for each non-empty set $\text{supp}(c)$ of agents, in which the leader bit of no agent will ever change again, is said to be *leader stable*. Let $L \subseteq A$ be the set of leaders in a leader stable configuration. A configuration is said to contain a *valid ordering* if it is leader stable and for any two leaders $a, b \in L$, $d_a \neq d_b$. Let $m_d(S)$ denote the number of leaders with different colors having label d in some leader stable configuration S . A label with $m_d(S) = 0$ in configuration S is called unoccupied, unused or free. Labels with $m_d(S) > 0$ are called used or occupied. Let $n_d(S) = \max(0, m_d(S) - 1)$. Intuitively n_d describes the number of “excess leaders” having label $d \in K$. For any $d \in K$ and any configuration S let:

$$g_d(S) = \min_{\substack{h \in K \\ m_h(S)=0}} (h - d) \mod k$$

$$R_d(S) = (k^2 + g_d(S)) \cdot n_d(S)$$

In case no $h \in K$ with $m_h(S) = 0$ exists, define $g_d(S) = 0$. For any configuration S further define:

$$R(S) = \sum_{d=0}^{k-1} R_d(S)$$

Intuitively $R(S)$ expresses “how far away” configuration S is from containing a valid ordering. For that $R_d(S)$ combines the values $n_d(S)$, expressing how many excess leaders are currently using label d , with the value $g_d(S)$, measuring the distance of label d to the next unoccupied label. $R_d(S)$ can therefore be thought of as (an upper bound on) the number of computation steps required by any interaction sequence starting in configuration S to ensure all excess leaders currently using label d are assigned some unused label. The factor k^2 takes into account that whenever a leader occupies a previously unoccupied label, the distance for all other labels to a free label increases. To readers familiar with ordinals it might help to think of the first uncountable ordinal ω instead of k^2 , expressing that an arbitrary increase in g_d is compensated by a decrement by one in n_d .

Step by step we now show that R will decrease until a configuration S with $R(S) = 0$ is reached, at which point a valid ordering exists and R will never increase again. First we show that $R(S) = 0$ corresponds to a valid ordering in S .

LEMMA 8.6. $R(S) = 0$ for a leader stable configuration $S \iff$ There is a valid ordering in configuration S .

PROOF. ($\implies$) Assume $R(S) = 0$ for some leader stable configuration S . By definition of n_d and g_d it holds that $n_d(S) \geq 0$ and $g_d(S) \geq 0 \forall d \in K$. The number of colors $k \geq 1$ and thus also k^2 is

a positive constant. Therefore $R(S) = 0$ implies $n_d(S) = 0 \forall d \in K$. For any label d there are therefore zero or one leaders having label d . This implies for any two leaders $a, b \in L$, $d_a \neq d_b$. S therefore contains a valid ordering.

($\impliedby$) Assume S contains a valid ordering. By definition S is leader stable. As it contains a valid ordering, there are no two agents $a, b \in L$, s.t. $d_a = d_b$. This means $m_d(S) < 2 \forall d \in K$, implying $n_d(S) = 0 \forall d \in K$ and thus $R(S) = 0$. $\square$

We show next that as long as the value of R is larger than zero there is some edge in the interaction graph that a scheduler might select, such that the value of R decreases. This is because as long as R is non-zero there are two leaders using the same label. Once they interact one of them selects a new label. Either it chooses one that no other leader is using or it at least gets closer to such an unoccupied label. Both options reduce R .

LEMMA 8.7. For any leader stable configuration S with $R(S) > 0$, there exists an interaction (a, b) , such that for configuration S' with $S \xrightarrow{(a,b)} S'$ it holds that $R(S') < R(S)$.

PROOF. For $R(S) > 0$ to hold there must be some $d \in K$, such that $n_d(S) > 0$. This implies $m_d(S) \geq 2$, meaning there are at least two leaders a and b using label d . Consider the interaction between a and b . In S they both share label d . The interaction causes a to update its label to $(d + 1) \mod k$ (for a shorter notation the $\mod k$ operation will be suppressed in the rest of the proof). Therefore $n_d(S') = n_d(S) - 1$, because in S' one leader less uses d than in S . There are two possible cases that can arise. Either the new label of a was unoccupied, meaning $m_{d+1}(S) = 0$ or it was already occupied, meaning $m_{d+1}(S) > 0$.

For convenience define the sum of excess leaders:

$$P(S) = \sum_{d'=0}^{k-1} n_{d'}(S)$$

Case $m_{d+1}(S) = 0$: Let $R(S) - R(S') = \Delta R$ be the difference between $R(S)$ and $R(S')$. Express ΔR as $\Delta R = \Delta R^+ - \Delta R^-$ with $\Delta R^+ \geq 0$ being the sum of all terms that decrease the value of $R(S')$ compared to $R(S)$, meaning they increase the difference between them, and with $\Delta R^- \geq 0$ being the sum of all terms that increase the value of $R(S')$ compared to $R(S)$, meaning they decrease the difference.

The assumption $m_{d+1}(S) = 0$ implies $m_{d+1}(S') = 1$ and therefore $n_{d+1}(S) = n_{d+1}(S') = 0$. In S no agent was using label $d + 1$ and in configuration S' only agent a is using it. Therefore in both S and S' there are no excess leaders in label d . As mentioned above $n_d(S') = n_d(S) - 1$. The value $n_{d''}$ for all other labels $d'' \in K$ remains unaffected by the interaction. Therefore $P(S') = P(S) - 1$, meaning the overall number of leaders that must select a different label decreases by one. This implies $\Delta R^+ \geq k^2$, because k^2 is added to the sum one time less and the difference between $R(S)$ and $R(S')$ is therefore increased by a value of at least k^2 .

Because $d + 1$ is not a free label anymore in S' , for every label $d' \in K$ the value of $g_{d'}(S')$ might increase compared to the value of $g_{d'}(S)$. Considering the range of $g_{d'}$ the value can increase by at most $k - 1$. With k many leaders this means $\Delta R^- \leq k(k - 1) = k^2 - k$,

because at most k times the value of $k - 1$ is added to the sum³, decreasing the difference in $R(S)$ and $R(S')$ by at most $k^2 - k$.

It now therefore holds that $\Delta R = \Delta R^+ - \Delta R^- \geq k^2 - (k^2 - k) = k > 1$. Therefore $R(S) > R(S')$.

Case $m_{d+1}(S) > 0$: In this case it holds that $n_{d+1}(S') = n_{d+1}(S) + 1$, meaning in S there already was at least one agent using label $d + 1$ and in configuration S' there is now one more. Together with $n_d(S') = n_d(S) - 1$ this means $P(S') = P(S)$, meaning the number of leaders that must select a new label remains unchanged.

We now argue that $\forall d' \in K : g_{d'}(S) = g_{d'}(S')$, meaning the distance to the next free label does not change for any label. It holds that $m_d(S) \geq 2$ and $m_d(S') \geq 1$. Also it holds that $m_{d+1}(S) \geq 1$ and $m_{d+1}(S') \geq 2$. Neither of the labels d and $d + 1$ was unoccupied in S and neither is in S' . For all other labels $d' \in K \setminus \{d, d + 1\} : m_{d'}(S) = m_{d'}(S')$. Interaction (a, b) thus did not lead to any previously occupied label being unoccupied or to any unoccupied label being occupied, therefore it did not change the distance to the next free label for any $d \in K$.

Because $R(S) > 0$ and because there are k many labels and at most k many leaders, there must be some free label. The fact that $m_d(S) > 0$ and $m_{d+1}(S) > 0$ implies $g_d(S) = g_{d+1}(S) + 1$. This means that because both labels d and $d + 1$ are used in S , the next free label is one step further away for d than it is for $d + 1$. Define $p = g_d(S)$. Then $g_{d+1}(S) = p - 1$ and as argued before $g_d(S') = p$ and $g_{d+1}(S') = p - 1$.

Now consider the following two terms:

$$\begin{aligned} R_d(S') + R_{d+1}(S') \\ &= n_d(S')(k^2 + p) + n_{d+1}(S')(k^2 + p - 1) \\ &= (n_d(S) - 1)(k^2 + p) + (n_{d+1}(S) + 1)(k^2 + p - 1) \end{aligned}$$

$$R_d(S) + R_{d+1}(S) = n_d(S)(k^2 + p) + n_{d+1}(S)(k^2 + p - 1)$$

Subtracting $R_d(S') + R_{d+1}(S')$ from $R_d(S) + R_{d+1}(S)$ gives:

$$\begin{aligned} R_d(S) + R_{d+1}(S) - (R_d(S') + R_{d+1}(S')) \\ &= k^2 + p - (k^2 + p - 1) = 1 \end{aligned}$$

It further holds that $\forall d' \in K \setminus \{d, d + 1\} : R_{d'}(S) = R_{d'}(S')$, because d' stays unaffected by the interaction. It overall follows that $R(S') < R(S)$. $\square$

As long as R is non-zero, there is some interaction that decreases R . We now show that for all configurations there is never any interaction that increases R .

LEMMA 8.8. *For any leader stable configuration S and interaction $S \xrightarrow{(a,b)} S'$ it holds that $R(S') \leq R(S)$.*

PROOF. Consider any operation of the transition function that may be executed on the interaction (a, b) .

Case $ic_a = ic_b$: Only the label of a non-leader may be changed in this case. Non-leaders do not influence the value of R in any way, implying $R(S) = R(S')$.

Case $ic_a \neq ic_b$: If $d_b \neq d_a$ or $ld_a = 0$ or $ld_b = 0$, then the state of

³This is very crude estimate, as obviously not for all k leaders the distance can be increased by $k - 1$ at the same time. A tighter analysis would allow to choose a constant smaller than k^2 in the definition of $R(S)$ but would otherwise have no other consequences for the proof.

a and b remains unchanged, meaning $R(S) = R(S')$. Otherwise d_a will be changed to $d'_a = (d_a + 1) \bmod k$. By Lemma 8.7 this implies $R(S') < R(S)$. $\square$

With all the above lemmata defined, we show correctness of ORDERING.

THEOREM 5.1 ($\star$). *ORDERING solves the ordering problem with $O(k^2)$ states.*

PROOF. After some number of interactions t_1 ORDERING reaches a leader stable configuration S_{t_1} according to Lemma 8.5. At that point $R(S_{t_1})$ can be defined. Obviously $R(S_{t_1})$ is a finite value. We show by induction over all possible values of $R(S_{t_1})$, that eventually a configuration S_{t_2} will be reached with $R(S_{t_2}) = 0$.

Induction base: $R(S_{t_1}) = 0$. In this case $t_1 = t_2$ and the claim obviously holds.

Induction hypothesis: ORDERING eventually reaches a configuration S_{t_2} with $R(S_{t_2}) = 0$, when at t_1 , $R(S_{t_1}) = x \geq 0$.

Induction step: To show: ORDERING eventually reaches a configuration S_{t_2} with $R(S_{t_2}) = 0$, when at t_1 , $R(S_{t_1}) = x + 1$.

According to Lemma 8.7 there exists some interaction (a, b) , such that $S_{t_1} \xrightarrow{(a,b)} S_{t_1+1}$ and $R(S_{t_1+1}) < R(S_{t_1})$. By Lemma 8.8 no interaction any scheduler might select increases R . Looking at the proof of Lemma 8.8 more closely it clearly also follows that any interaction that does not decrease R , does not change the state of any leader. Therefore no matter how often a scheduler may select any interaction that does not decrease R , leading to some configuration S'_{t_1} , the interaction (a, b) still produces a configuration S'_{t_1+1} such that $S'_{t_1} \xrightarrow{(a,b)} S'_{t_1+1}$ and $R(S'_{t_1+1}) < R(S'_{t_1}) = R(S_{t_1})$. A weakly fair scheduler (as well as a random or a globally fair one) will therefore eventually either select interaction (a, b) or another one reducing R . Therefore eventually R will be decreased and by induction hypothesis a configuration S_{t_2} with $R(S_{t_2}) = 0$ will be reached.

Obviously R can never get negative. Using Lemma 8.8 again therefore after t_2 , R will never increase and thus remain zero. In S_{t_2} there is a valid ordering on the leaders, according to Lemma 8.6. After t_2 the only state changes that will occur are those of non-leaders interacting with the leaders of their color, causing the non-leader to copy the label of the leader, until finally in some configuration S_{t_3} , all agents of the same color agree on the same binary label, which is different from the labels of all other colors.

ORDERING stores the initial color of an agent, for which there are k options at most, the label associated with that color, again having at most k options, and a single bit indicating whether the agent is a leader. This gives $2k^2 \in O(k^2)$ possible states. $\square$

5.7 Proof of Theorem 5.2

THEOREM 5.2 ($\star$). *ORD-CIRCLES solves relative majority in the unordered setting against a weakly-fair scheduler using $O(k^4)$ states. This can be reduced to $O(k^3)$ many states if every agent is only required to report whether their color wins or loses.*

We will first show properties of ORD-CIRCLES, before ultimately arguing for its correctness. On a high level the proof works as follows. Every operation the ordering step intends to make on an agent's bra is eventually carried out and corresponds precisely to ordering protocol ORDERING. The steps are carried out after a finite number of interactions and in a way that preserves correctness of sub-protocol CIRCLES. Once the ordering converges only CIRCLES is run and the overall process stabilizes.

We subdivide this argument further into smaller parts. First we show that ORD-CIRCLES preserves CIRCLES's invariant. This is crucial to later argue for the overall convergence of ORD-CIRCLES

LEMMA 8.9. *ORD-CIRCLES maintains Invariant 3.3, meaning in ORD-CIRCLES for every $i \in K$, there are as many bras $\langle i |$ in the population, as there are kets $|i \rangle$.*

PROOF. Step (1) does not change the bracket of any agent and therefore does not influence the invariant. Step (5) updates the bracket of agents according to CIRCLES, which preserves the invariant. Left to consider are therefore steps (2), (4) and (3).

Step (2) re-initializes pending agent a . Any pending agent is consistent, because of the following. Step (4) is the only way for an agent to be put into ordering state CL . For that it must be consistent and in ordering state CLW . Only non-leaders are put into ordering state CLW in step (1). An agent in ordering state CL , being a non-leader, only interacts using step (2), which re-initializes it. Therefore as long it is pending, it remains consistent. The bracket of a therefore has the form $\langle i_a | i_a \rangle$ for some i_a . After re-initializing a stores $\langle i_b | i_b \rangle$. Step (2) therefore removes a matching bracket pair and adds a matching bracket pair to the population. This clearly preserves Invariant 3.3.

Step (4) re-initializes consistent agent a with bracket $\langle i_a | i_a \rangle$ to $\langle (i_a + 1) \bmod k | (i_a + 1) \bmod k \rangle$. As before one matching bracket pair is removed and one added, preserving the invariant.

Step (3) exchanges the kets of two agents, obviously preserving the Invariant. The overall claim therefore holds. $\square$

Next we consider the leader election process within the ordering routine to simplify subsequent proofs.

LEMMA 8.10. *After some number of interactions there is a unique leader in every set of agents initially supporting the same color. Afterwards the leader bit of no agent changes.*

PROOF. See proof of Lemma 8.5 $\square$

We again say a configuration is *leader stable* if it contains a unique leader per color in the population and the leader bit of no agent can change anymore. Once such a configuration is reached, further important observations can be made about the population.

LEMMA 8.11. *After a leader stable configuration is reached, the ordering state of any agent changes only finitely often.*

PROOF. Again we call some label $i \in K$ *used* in case there exists a leader that stores the label as its bra and *free* otherwise. A leader can never be in ordering state CL or CLW . To put a leader a into ordering state $os_a = I$, it must interact with some leader b with the same bra and $os_b = U$. In this interaction ordering state and bra of b remain unchanged. Only leaders with ordering state I ever change their bra. Therefore once a label is used by some leader, it

will never be free, because two leaders using the label are required to put one of them into ordering state I , while ordering state and bra of the other one remain unaffected by the interaction.

Now assume there was some leader a that was changed from ordering state U to ordering state I k many times. Then a must have increased its label $k - 1$ times, as a leader can go from ordering state I to ordering state U only using step (4). This means a used every label once at some point. As described above no label that was once used by some leader will ever become free again. Therefore all the labels are used. However, a goes into ordering state I a k^{th} time, meaning there are at least two leaders in the last label a chose. Overall this means for every label there is at least one leader using it and there is a label that at least two leaders use. This means there are more than k leaders, which is a contradiction, because, as shown before, in a leader stable configuration there are as many leaders as there are colors and there are at most k colors. This contradiction implies that leaders only go into ordering state I at most $k - 1$ times and therefore only change their bra at most $k - 1$ times. Since leaders can only alternate between ordering states U and I , they therefore change their ordering state only finitely often.

Some non-leader a goes from ordering state U into CLW only in case the leader b of its color changed its label since a last interacted with it. As just shown leaders change their label at most $k - 1$ many times and therefore non-leaders go into state CLW at most $k - 1$ times and therefore also only update their label at most $k - 1$ times. Non-leaders can go from ordering state U to CLW to CL to U again, but not backwards. They therefore only update their states finitely often. $\square$

The argument above is crucial to now argue that agents do not wait indefinitely to become consistent. This is important because only consistent agents can be re-initialized with a new bra.

LEMMA 8.12. *Any inconsistent and unstable agent a eventually becomes consistent.*

PROOF. Let a be an inconsistent and unstable agent. This means the bracket of a contains $\langle i_a | j_a \rangle$ with $i_a \neq j_a$. For contradiction assume a remains inconsistent throughout the computation. By Invariant 3.3, there exists some agent b with bracket $\langle i_b | j_b \rangle$, such that $i_b \neq j_b = i_a$. For every configuration let B be the set of non-pending agents with i_a as ket. We will now argue that under a weakly fair scheduler a eventually interacts with an agent from B .

By weak fairness a interacts with all other agents infinitely often. Also, as long as a is inconsistent, B always contains some other agent. According to Lemma 8.11 no agent changes its ordering state infinitely often. Clearly this also holds in case we assume a (and potentially other agents) to remain inconsistent throughout the computation, because in case a is a non-leader it cannot make other agents unstable and in case a is a leader, it cannot make leaders of different color unstable (since for that it would need to be stable) and can make non-leaders of its color unstable only once (since it does not change its *bra*). This means after some number of interactions either a interacted with some agent in B or a remains inconsistent, but none of the other agents change their ordering state anymore. All agents then only interact using step (5) of ORD-CIRCLES, as all other steps update the ordering state of at least one agent. By correctness of CIRCLES therefore eventually the bracket of all agents

stabilizes, meaning no agents exchange their kets anymore (even in case pending vertices exist as they are consistent and can safely be ignored). At this point weak fairness guarantees that a interacts with some agent in B . Step (3) will be executed, making a consistent, which produces a contradiction. No other step of ORD-CIRCLES can change os_a before a is consistent. $\square$

With all the above arguments in place we can now argue that the sub-protocol ORDERING actually converges within ORD-CIRCLES.

LEMMA 8.13. *After some number of interactions for all pairs of agents $a, b \in A$ it holds that $\langle i_a | = \langle i_b | \iff ic_a = ic_b$. All agents are in ordering state $os = U$. Both bra and os of no agent subsequently changes.*

PROOF. Step (1) puts a leader into ordering state I exactly in case ORDERING would increment its label, namely in case there is another (stable) leader with of different color using the same bra. Whenever an agent is put into ordering state I , then it either is consistent or by Lemma 8.12 eventually will be. At that point step (4) carries out the increment modulo k and puts the agent back into ordering state U . Step (1) puts a non-leader into ordering state CLW exactly in case ORDERING would copy the label of its leader, namely in case its leader changed its bra since they last interacted (and the agent is not already trying to carry out a copy operation). Again either the agent already is consistent or it eventually will be, at which point step (4) and then step (2) ensure it copies its leaders label and sets its ordering state to U .

By correctness of the ordering protocol ORDERING, therefore eventually for two agents $a, b \in A$ it holds that $\langle i_a | = \langle i_b |$ if and only if they store the same initial color. On the last increment or copy operation all agents set $os = U$. With all agents of the same initial color sharing the same bra, no agent can subsequently be put into ordering state CLW and therefore also not into ordering state CL . With all agents of different color having different bras, no agent can be put into ordering state I . Therefore all agents store ordering state U . $\square$

Having shown that an ordering on the colors is eventually established, we now arrive at the final theorem.

THEOREM 5.2 (★). *ORD-CIRCLES solves relative majority in the unordered setting against a weakly-fair scheduler using $O(k^4)$ states. This can be reduced to $O(k^3)$ many states if every agent is only required to report whether their color wins or loses.*

PROOF. According to Lemma 8.13 after some number t_1 of interactions, the ordering process converges, $os = U$ for all agents and $\langle i_a | = \langle i_b | \iff ic_a = ic_b$. At that point only step (5) of ORD-CIRCLES is executed, meaning only CIRCLES is used. Because Invariant 3.3 of CIRCLES is maintained, according to Lemma 8.9, CIRCLES converges for the final distribution of bras. This means after some number $t_2 \geq t_1$ of interactions all agents agree on the bra in relative majority and therefore on the initial color in relative majority. As described before the output can be adapted to contain a color instead of a label.

For some agent a , ORD-CIRCLES stores ic_a , $\langle i_a |_{j_a}$, out_a , ld_a , os_a and some number of bits to handle reporting, sharing or breaking of ties. The values ic_a , i_a , j_a and out_a can take k values each. All

values ld_a , os_a and the bits for breaking, sharing and reporting ties can take a constant number of different values. This overall gives $O(k^4)$ possible states. Removing out and adding a single bit storing whether an agent is in relative majority or not, this can be reduced to $O(k^3)$ for versions with binary win-lose output. $\square$

8.8 Proof of Lemma 6.5

PROOF. Rule (2) cannot modify any agents as there are no two tokens on kets with the same color. Rule (3) cannot modify any agents as there are no two tokens associated to the same ranking. Rule (4) cannot modify any agents as for every i , all bras and kets have the same ranking $r(i)$. Rule (5) cannot modify any agents by Lemma 6.2. Rule (6) cannot modify any agents as the token for i with $r(i) = 0$ is on a bracket $\langle i, r(i) = 0 | r(i) = 0, i \rangle^*$ which minimizes the ranking in the bra, while for all $j \neq i$, we have that the kets containing j are all of the same form: $\langle \ell, r(\ell) | r(j) = r(\ell) + 1, j \rangle$, since a token is not transferred between agents of same bracket. Finally, Rule (7) cannot modify any agents, as there are two tokens on brackets with bra of ranking 0 one of which is of the form $\langle i, r(i) = 0 | r(i) = 0, i \rangle^*$, while all others are on brackets of bras of different weights. The two tokens on the same bra cannot interact because one of them has the same bra and ket, while any other pair of tokens cannot interact because they have different bras. $\square$

8.9 Proof of Lemma 6.12

PROOF. In the particular configuration $E_{\ell-1}$, none of the rules (2) to (5) apply. It thus remains to show how Rules (6) and (7) are useful.

We know that in configuration $E_{\ell-1}$, the $\ell - 1$ most popular colors have the correct assigned ranking, that is, $|r(j), j\rangle^*$ exists in the configuration for all $j \leq \ell - 1$, and the CIRCLES protocol has stabilized. Let u be the ℓ -th most popular color, and v be the one assigned rank ℓ in $E_{\ell-1}$. If $u = v$, then we are already in configuration E_ℓ and there is nothing to prove. We thus assume that $u \neq v$.

Let $i_1, \dots, i_{\ell-1}$ be the $\ell - 1$ most popular colors. Since the CIRCLES protocol is stabilized, we know that there is a greedy independent set with the elements $i_1, \dots, i_{\ell-1}, u$.

- If $\ell = 0$, then this is simply one element, and there is a bracket $\langle u, u | u, u \rangle^\circ$. Through Rule (6), this bracket can get the token for u , and through Rule (7), can obtain a newly assigned ranking of 0.
- Otherwise, if $\ell \neq 0$, since $x_{i_{\ell-1}} > x_u$, we have that every greedy independent set containing u also contains $i_{\ell-1}$. Therefore, coupled with the fact that there exists a greedy independent set with $i_1, \dots, i_{\ell-1}, u$, the bra with smallest ranking associated to a ket of the form $|\cdot, u\rangle$ is $\langle i_{\ell-1}, \ell - 1 | \cdot, u \rangle$. Through Rule (6), this bracket $\langle i_{\ell-1}, \ell - 1 | \cdot, u \rangle$ can get the token for u . Similarly, we have that every greedy independent set containing v also contains $i_{\ell-1}$. Therefore, coupled with the fact that there exists a greedy independent set with $i_1, \dots, i_{\ell-1}, v, \dots$, the bra with smallest ranking associated to a ket of the form $|\ell, v\rangle$ is $\langle i_{\ell-1}, \ell - 1 | \ell, v \rangle$. Through Rule (6), this bracket $\langle i_{\ell-1}, \ell - 1 | \ell, v \rangle$ can get the token for v . And thus through Rule (7), $\langle i_{\ell-1}, \ell - 1 | \cdot, u \rangle^*$ and $\langle i_{\ell-1}, \ell - 1 | \ell, v \rangle^*$ meet and exchange rankings, with u getting the ranking $\ell = r(u)$.

The tokens who have had their rankings changed can now broadcast the new value of their rankings through Rule (4), and then any

steps needed to stabilize the CIRCLES protocol can happen through Rule (5). $\square$